Universidad El Bosque

Programa: Ingeniería de Sistemas

Asignatura: Bases de Datos 2

Docente: Ing., Esp., M.Ed. Christian Felipe Duarte

TALLER PL/SQL

Evidencias de ejecución y explicación de la solución

Integrante 1	David Bejarano Montes
Integrante 2	Samuel Camelo Araque
Integrante 3	Samuel Novoa Martinez
Fecha de entrega	03/10/2026
Base de datos	Oracle Database 19c — esquema SCAMELOA
Archivo de código	TALLER_PLSQL_GRUPO_<n>.sql

Parte 1 — Procedimientos almacenados (8 puntos)

1.1 SP_REGISTRAR_PRODUCTO — 2 puntos

Enunciado: *Registra un producto nuevo con id de SEQ_PRODUCTO, valida precio > costo y categoría existente, y maneja el código duplicado.*

Explicación de la solución

El procedimiento valida primero las reglas de negocio (precio de venta mayor que el costo) y luego que la categoría exista mediante un COUNT, y solo después consume SEQ_PRODUCTO.NEXTVAL: así una solicitud inválida no gasta números de la secuencia. El identificador generado se devuelve por el parámetro OUT. Cada causa de rechazo tiene su propia excepción y su código (-20001 precio, -20002 categoría, -20003 código duplicado), de modo que quien llama puede distinguirlas.

El código duplicado se captura con DUP_VAL_ON_INDEX, que Oracle levanta al violarse UK_PRODUCTO_CODIGO. Límite operacional: esa excepción no indica cuál índice único falló; por eso la secuencia debe estar sincronizada con los ids existentes (1001–1015); de lo contrario, una colisión de llave primaria se reportaría erróneamente como código duplicado.

Evidencias de ejecución

Captura 1.1-a — Registro exitoso y consulta del producto creado

CAPTURA 1.1-a

```
SET SERVEROUTPUT ON;

DECLARE
    vn_id NUMBER;
BEGIN
    SP_REGISTRAR_PRODUCTO(
        param_codigo       => 'TEST-001',
        param_nombre       => 'Producto de prueba',
        param_id_categoria => 1,
        param_precio       => 50000,
        param_costo        => 30000,
        param_stock_minimo => 10,
        param_id_producto  => vn_id
    );

    DBMS_OUTPUT.PUT_LINE('Producto creado con ID: ' || vn_id);
END;
/
```

Script Output | Script Output 1 | Query Result | Query Re

Task completed in 0.189 seconds

```
Producto creado con ID: 1016


PL/SQL procedure successfully completed.
```

```
SELECT id_producto,
       codigo,
       nombre,
       id_categoria,
       precio_unitario,
       costo_unitario,
       stock_minimo,
       activo
FROM PRODUCTO
WHERE codigo = 'TEST-001';
```

Script Output | Script Output 1 | Query Result | Query Result 1 | Query Result 2 | Query Result 3

SQL | All Rows Fetched: 1 in 0.08 seconds

	ID_PRODUCTO	CODIGO	NOMBRE	ID_CATEGORIA	PRECIO_UNITARIO	COSTO_UNITARIO	STOCK_MINIMO	ACTIVO
1	1016	TEST-001	Producto de prueba	1	50000	30000	10	S

Captura 1.1-b — Rechazos: precio inválido, categoría inexistente y código duplicado

CAPTURA 1.1-b

```
DECLARE
    vn_id NUMBER;
BEGIN
    SP_REGISTRAR_PRODUCTO(
        param_codigo       => 'TEST-ERR-01',
        param_nombre       => 'Producto precio invalido',
        param_id_categoria => 1,
        param_precio       => 20000,
        param_costo        => 30000,
        param_stock_minimo => 5,
        param_id_producto  => vn_id
    );
END;
/
```

Script Output | Script Output 1 | Query Result | Query Res

Task completed in 0.279 seconds

```
*
ERROR at line 1:
ORA-20001: El precio de venta debe ser mayor que el costo unitario
ORA-06512: at "SCAMELOA.SP_REGISTRAR_PRODUCTO", line 62
ORA-06512: at line 4
```

```
DECLARE
    vn_id NUMBER;
BEGIN
    SP_REGISTRAR_PRODUCTO(
        param_codigo       => 'TEST-ERR-02',
        param_nombre       => 'Producto categoria invalida',
        param_id_categoria => 99,
        param_precio       => 50000,
        param_costo        => 30000,
        param_stock_minimo => 5,
        param_id_producto  => vn_id
    );
END;
/
```

Script Output | Script Output 1 | Query Result | Query

Task completed in 0.282 seconds

```
*
ERROR at line 1:
ORA-20002: La categoria indicada no existe
ORA-06512: at "SCAMELOA.SP_REGISTRAR_PRODUCTO", line 69
ORA-06512: at line 4
```

```
DECLARE
    vn_id NUMBER;
BEGIN
    SP_REGISTRAR_PRODUCTO(
        param_codigo       => 'TEST-001',
        param_nombre       => 'Producto duplicado',
        param_id_categoria => 1,
        param_precio       => 60000,
        param_costo        => 30000,
        param_stock_minimo => 5,
        param_id_producto  => vn_id
    );
END;
/
```

Script Output | Script Output 1 | Query Result

Task completed in 0.295 seconds

```
*
ERROR at line 1:
ORA-20003: Ya existe un producto con el codigo TEST-001
ORA-06512: at "SCAMELOA.SP_REGISTRAR_PRODUCTO", line 76
ORA-06512: at line 4
```

1.2 SP_AJUSTAR_STOCK — 3 puntos

Enunciado: *Registra ENTRADA, SALIDA o AJUSTE sobre una bodega, actualiza EXISTENCIA, inserta el movimiento y devuelve el saldo; la lectura debe protegerse de la concurrencia.*

Explicación de la solución

Valida que el tipo sea ENTRADA, SALIDA o AJUSTE y que la cantidad sea positiva. Luego lee la existencia con SELECT … FOR UPDATE: el bloqueo de la fila impide que otra sesión modifique el saldo entre la lectura y la escritura (pérdida de actualización), que es la protección que pide el enunciado. ENTRADA suma y, si la combinación bodega-producto no existe (NO_DATA_FOUND), la crea; SALIDA y AJUSTE sobre una combinación inexistente se rechazan (-20006); la SALIDA compara la cantidad con la existencia antes de restar (-20007), por lo que el saldo nunca queda negativo; AJUSTE fija el saldo al valor indicado.

En todos los casos se inserta el movimiento con SEQ_MOVIMIENTO y el saldo resultante se devuelve por el parámetro OUT. Límite operacional: cuando dos sesiones hacen la primera ENTRADA sobre una combinación que aún no existe, no hay fila que bloquear y la segunda recibiría un error de llave primaria.

Evidencias de ejecución

Captura 1.2-a — ENTRADA, SALIDA y AJUSTE sobre una existencia existente (1001, bodega 1)

CAPTURA 1.2-a

```
DECLARE
    vn_saldo NUMBER;
BEGIN
    SP_AJUSTAR_STOCK(
        param_id_producto => 1001,
        param_id_bodega   => 1,
        param_tipo        => 'ENTRADA',
        param_cantidad    => 10,
        param_observacion => 'Evidencia 1.2-a - Entrada',
        param_saldo_final => vn_saldo
    );
    DBMS_OUTPUT.PUT_LINE(
        'Saldo despues de ENTRADA: ' || vn_saldo
    );
    SP_AJUSTAR_STOCK(
        param_id_producto => 1001,
        param_id_bodega   => 1,
        param_tipo        => 'SALIDA',
        param_cantidad    => 5,
        param_observacion => 'Evidencia 1.2-a - Salida',
        param_saldo_final => vn_saldo
    );
    DBMS_OUTPUT.PUT_LINE(
        'Saldo despues de SALIDA: ' || vn_saldo
    );
    SP_AJUSTAR_STOCK(
        param_id_producto => 1001,
        param_id_bodega   => 1,
        param_tipo        => 'AJUSTE',
        param_cantidad    => 100,
        param_observacion => 'Evidencia 1.2-a - Ajuste',
        param_saldo_final => vn_saldo
    );
    DBMS_OUTPUT.PUT_LINE(
        'Saldo despues de AJUSTE: ' || vn_saldo
```

Script Output | Script Output 1 | Query Result | Query Re

Task completed in 0.184 seconds

```
PL/SQL procedure successfully completed.

Saldo despues de ENTRADA: 110
Saldo despues de SALIDA: 105
Saldo despues de AJUSTE: 100
```

Captura 1.2-b — ENTRADA que crea la combinación bodega-producto (1015, bodega 4)

CAPTURA 1.2-b

```
SELECT id_movimiento,
       id_producto,
       id_bodega,
       tipo,
       cantidad,
       fecha_mov,
       observacion
FROM MOVIMIENTO_INVENTARIO
WHERE id_producto = 1015
  AND id_bodega = 4
  AND observacion = 'Evidencia 1.2-b – Nueva existencia'
ORDER BY id_movimiento DESC;
```

Script Output | Script Output 1 | Query Result | Query Result 1 | Query Result 2 | Query Result 3

SQL | All Rows Fetched: 1 in 0.086 seconds

	ID_MOVIMIENTO	ID_PRODUCTO	ID_BODEGA	TIPO	CANTIDAD	FECHA_MOV	OBSERVACION
1	42	1015	4	ENTRADA	25	04/10/26	Evidencia 1.2-b – Nueva existencia

Captura 1.2-c — Rechazos: SALIDA inexistente, SALIDA mayor a la existencia, tipo y cantidad inválidos

CAPTURA 1.2-c

```
DECLARE
    vn_saldo NUMBER;
BEGIN
    SP_AJUSTAR_STOCK(
        param_id_producto => 1015,
        param_id_bodega   => 2,
        param_tipo        => 'SALIDA',
        param_cantidad    => 5,
        param_observacion => 'Prueba existencia inexistente',
        param_saldo_final => vn_saldo
    );
END;
/

DECLARE
```

Script Output | Script Output 1 | Query Result | Query Result

Task completed in 0.295 seconds

```
*
ERROR at line 1:
ORA-20006: No existe inventario para el producto en la bodega indicada
ORA-06512: at "SCAMELOA.SP_AJUSTAR_STOCK", line 146
ORA-06512: at line 4
```

```
DECLARE
    vn_saldo NUMBER;
BEGIN
    SP_AJUSTAR_STOCK(
        param_id_producto => 1001,
        param_id_bodega   => 1,
        param_tipo        => 'SALIDA',
        param_cantidad    => 999,
        param_observacion => 'Prueba stock insuficiente',
        param_saldo_final => vn_saldo
    );
END;
/

DECLARE
```

Script Output | Script Output 1 | Query Result | Qu

Task completed in 0.283 seconds

```
*
ERROR at line 1:
ORA-20007: La salida supera la existencia disponible
ORA-06512: at "SCAMELOA.SP_AJUSTAR_STOCK", line 153
ORA-06512: at line 4
```

```
DECLARE
    vn_saldo NUMBER;
BEGIN
    SP_AJUSTAR_STOCK(
        param_id_producto => 1001,
        param_id_bodega   => 1,
        param_tipo        => 'VENTA',
        param_cantidad    => 5,
        param_observacion => 'Prueba tipo invalido',
        param_saldo_final => vn_saldo
    );
END;
/
```

Script Output | Script Output 1 | Query Result | Query

Task completed in 0.284 seconds

```
*
ERROR at line 1:
ORA-20004: El tipo de movimiento debe ser ENTRADA, SALIDA o AJUSTE
ORA-06512: at "SCAMELOA.SP_AJUSTAR_STOCK", line 132
ORA-06512: at line 4
```

```
DECLARE
    vn_saldo NUMBER;
BEGIN
    SP_AJUSTAR_STOCK(
        param_id_producto => 1001,
        param_id_bodega   => 1,
        param_tipo        => 'ENTRADA',
        param_cantidad    => 0,
        param_observacion => 'Prueba cantidad invalida',
        param_saldo_final => vn_saldo
    );
END;
/
```

Script Output | Script Output 1 | Query Result | Qu

Task completed in 0.282 seconds

```
*
ERROR at line 1:
ORA-20005: La cantidad debe ser mayor que cero
ORA-06512: at "SCAMELOA.SP_AJUSTAR_STOCK", line 139
ORA-06512: at line 4
```

```
SELECT id_producto,
       id_bodega,
       cantidad
FROM EXISTENCIA
WHERE id_producto = 1001
  AND id_bodega = 1;
```

Script Output | Script Output 1 | Query Re

SQL | All Rows Fetched: 1 in 0.077 sec

	ID_PRODUCTO	ID_BODEGA	CANTIDAD
1	1001	1	120

1.3 SP_CREAR_PEDIDO — 3 puntos

Enunciado: *Crea la cabecera de un pedido en BORRADOR y devuelve el id; rechaza clientes inactivos y sin cupo con mensajes distinguibles.*

Explicación de la solución

Consulta estado, cupo y saldo del cliente en una sola sentencia y aplica las validaciones en un orden fijo: existencia del cliente, estado activo y cupo disponible (cupo menos saldo pendiente, que debe ser mayor que cero). Cada causa tiene su excepción, código y mensaje propios (-20008 inexistente, -20009 inactivo, -20010 sin cupo), de modo que los dos rechazos exigidos por el enunciado son distinguibles entre sí. Si todo es válido toma el id de SEQ_PEDIDO e inserta la cabecera en estado BORRADOR, con total 0 y fecha SYSDATE, devolviendo el identificador por el parámetro OUT.

Evidencias de ejecución

Captura 1.3-a — Creación de un pedido en BORRADOR (cliente 501)

CAPTURA 1.3-a

```
DECLARE
    vn_id_pedido NUMBER;
BEGIN

    SP_CREAR_PEDIDO(
        param_id_cliente => 501,
        param_id_pedido  => vn_id_pedido
    );

    DBMS_OUTPUT.PUT_LINE(
        'Pedido creado con ID: ' || vn_id_pedido
    );

END;
/
```

Script Output | Script Output 1 | Query Result

Task completed in 0.198 seconds

```
Pedido creado con ID: 10


PL/SQL procedure successfully completed.
```

```
SELECT id_pedido,
       id_cliente,
       fecha_pedido,
       estado,
       total
FROM PEDIDO
WHERE id_cliente = 501
ORDER BY id_pedido DESC;
```

Script Output | Script Output 1 | Query Result | Query Resul

SQL | All Rows Fetched: 5 in 0.085 seconds

	ID_PEDIDO	ID_CLIENTE	FECHA_PEDIDO	ESTADO	TOTAL
1	10	501	04/10/26	BORRADOR	0
2	9	501	04/10/26	BORRADOR	0
3	6	501	05/09/26	BORRADOR	0
4	2	501	02/07/26	CONFIRMADO	641000
5	1	501	10/06/26	CONFIRMADO	1286500

Captura 1.3-b — Rechazos: cliente inactivo, sin cupo e inexistente

CAPTURA 1.3-b

```
DECLARE
    vn_id_pedido NUMBER;
BEGIN

    SP_CREAR_PEDIDO(
        param_id_cliente => 506,
        param_id_pedido  => vn_id_pedido
    );

END;
/
```

Script Output | Script Output 1 | Query Re

Task completed in 0.28 seconds

```
DECLARE
*
ERROR at line 1:
ORA-20009: El cliente se encuentra inactivo
ORA-06512: at "SCAMELOA.SP_CREAR_PEDIDO", line 79
ORA-06512: at line 5
```

```
DECLARE
    vn_id_pedido NUMBER;
BEGIN

    SP_CREAR_PEDIDO(
        param_id_cliente => 505,
        param_id_pedido  => vn_id_pedido
    );

END;
/
```

Script Output | Script Output 1 | Query Result

Task completed in 0.284 seconds

```
*
ERROR at line 1:
ORA-20010: El cliente no tiene cupo de credito disponible
ORA-06512: at "SCAMELOA.SP_CREAR_PEDIDO", line 86
ORA-06512: at line 5
```

```
DECLARE
    vn_id_pedido NUMBER;
BEGIN

    SP_CREAR_PEDIDO(
        param_id_cliente => 9999,
        param_id_pedido  => vn_id_pedido
    );

END;
/
```

Script Output | Script Output 1 | Query Resu

Task completed in 0.278 seconds

```
*
ERROR at line 1:
ORA-20008: El cliente indicado no existe
ORA-06512: at "SCAMELOA.SP_CREAR_PEDIDO", line 72
ORA-06512: at line 5
```

Parte 2 — Funciones (8 puntos)

2.1 FN_STOCK_DISPONIBLE — 2 puntos

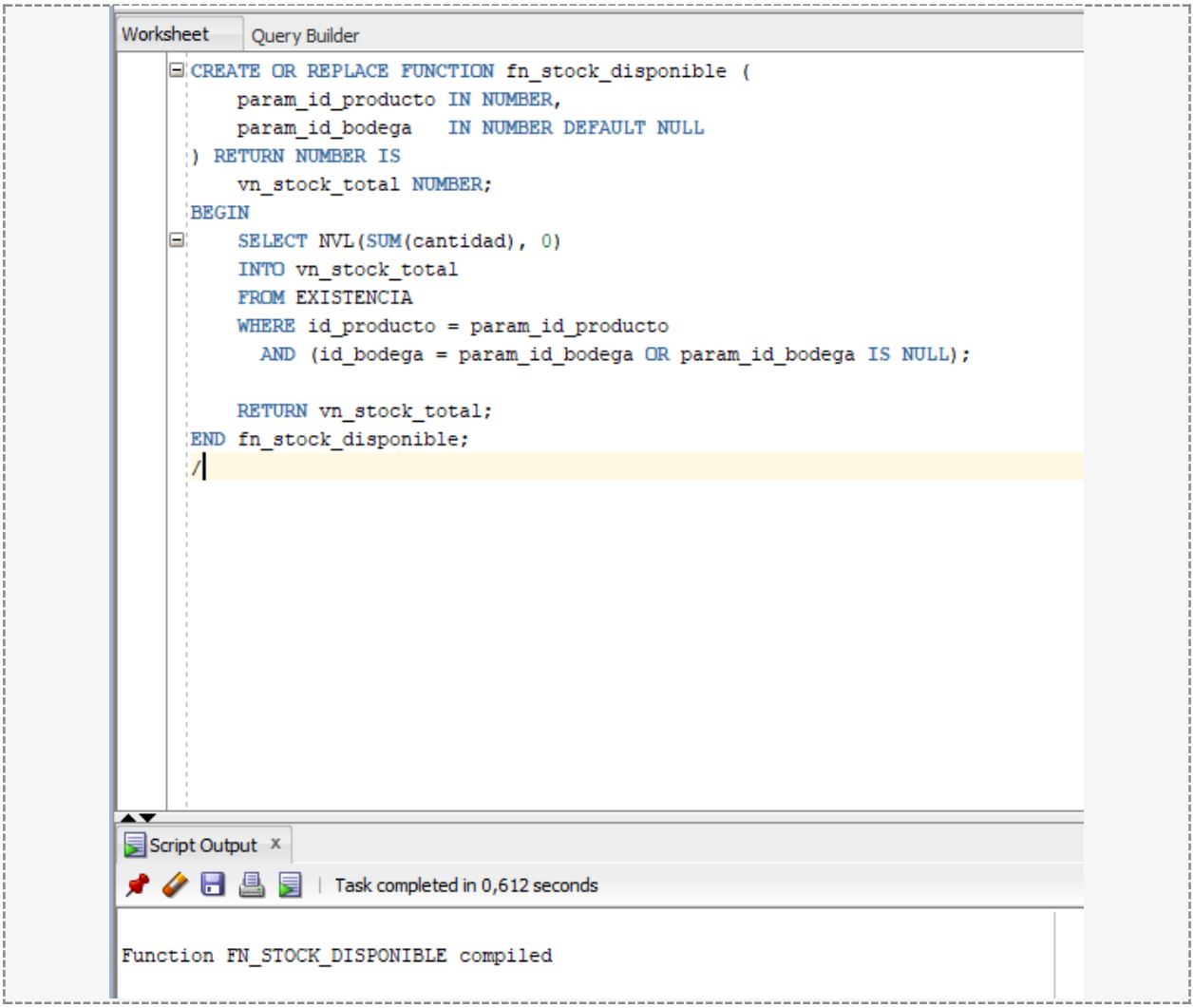
Enunciado: *Cantidad disponible de un producto: por bodega o consolidada si la bodega es nula; un producto inexistente retorna 0.*

Explicación de la solución

Una sola consulta con SUM y NVL cubre todos los casos: con bodega devuelve el saldo de esa bodega, con bodega nula el consolidado, y un producto sin existencias o inexistente devuelve 0 sin excepción, porque una función de grupo sin GROUP BY siempre retorna una fila (con NULL si no hay coincidencias) y NVL la convierte en 0. El predicado (id_bodega = param OR param IS NULL) evita escribir dos consultas; su costo es que puede impedir un plan con índice por bodega, irrelevante con este volumen.

Evidencias de ejecución

Captura 2.1 — FN_STOCK_DISPONIBLE: bodega, consolidado, sin existencias e inexistente



2.2 FN_CUPO_DISPONIBLE — 2 puntos

Enunciado: *Cupo de crédito disponible (cupo − saldo); cliente inactivo retorna 0 y el resultado nunca es negativo.*

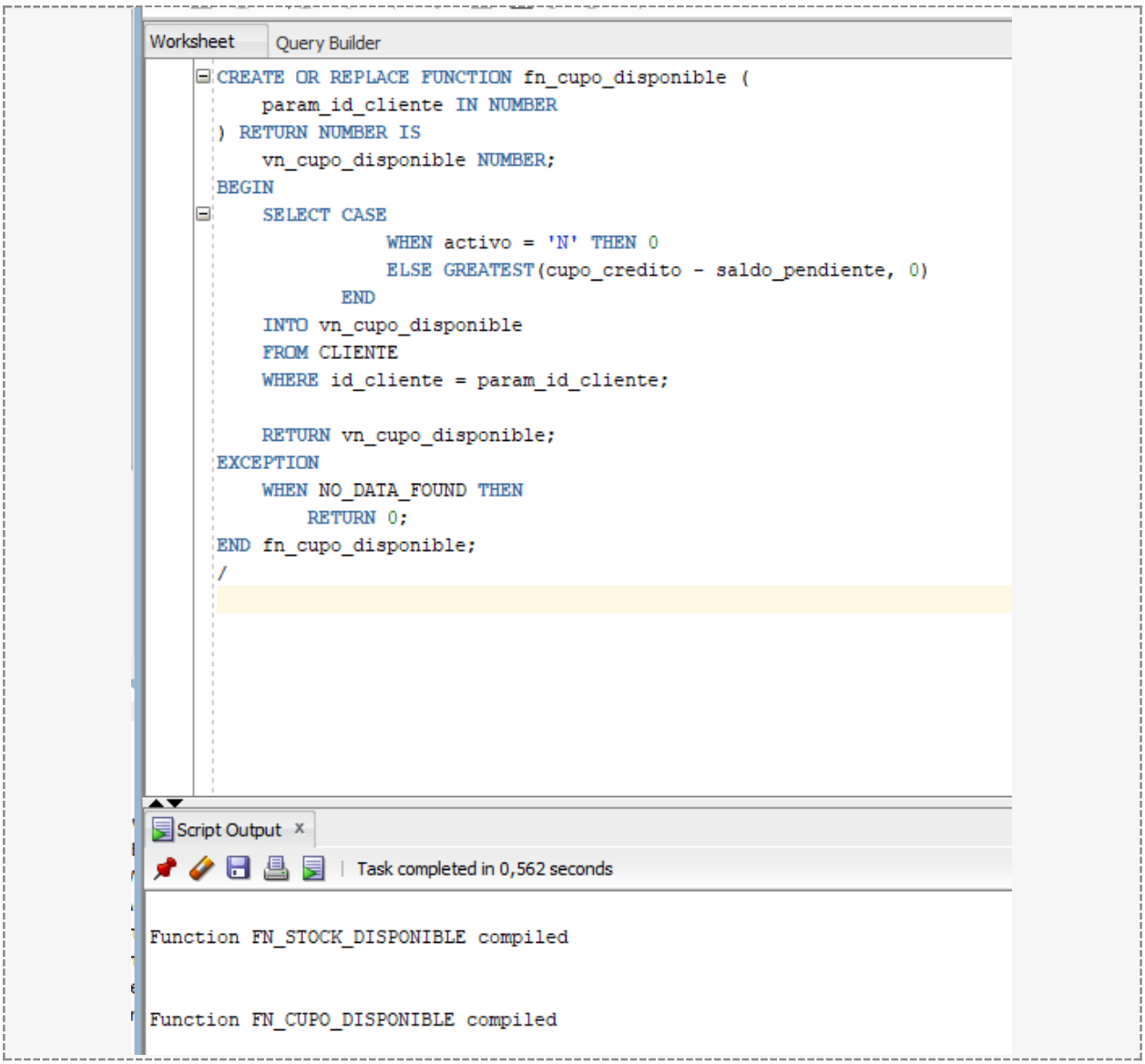
Explicación de la solución

El motor SQL resuelve las dos reglas: CASE devuelve 0 para clientes inactivos y GREATEST(cupo − saldo, 0) garantiza que el resultado nunca sea negativo, sin IF anidados en PL/SQL ni consultas adicionales. Un cliente inexistente devuelve 0 (NO_DATA_FOUND capturado) en lugar de propagar una excepción, para que la función sea segura al usarse dentro de consultas sobre muchas filas.

Evidencias de ejecución

Captura 2.2 — FN_CUPO_DISPONIBLE para todos los clientes y para un cliente inexistente

Ejecutar: evidencias_pruebas.sql › E-2.2



2.3 FN_CALCULAR_DESCUENTO — 3 puntos

Enunciado: *Valor en pesos del descuento: 8 % desde 2.000.000, 5 % entre 800.000 y 1.999.999, 0 % por debajo, +2 % si el cliente no tiene saldo pendiente, tope acumulado 10 %.*

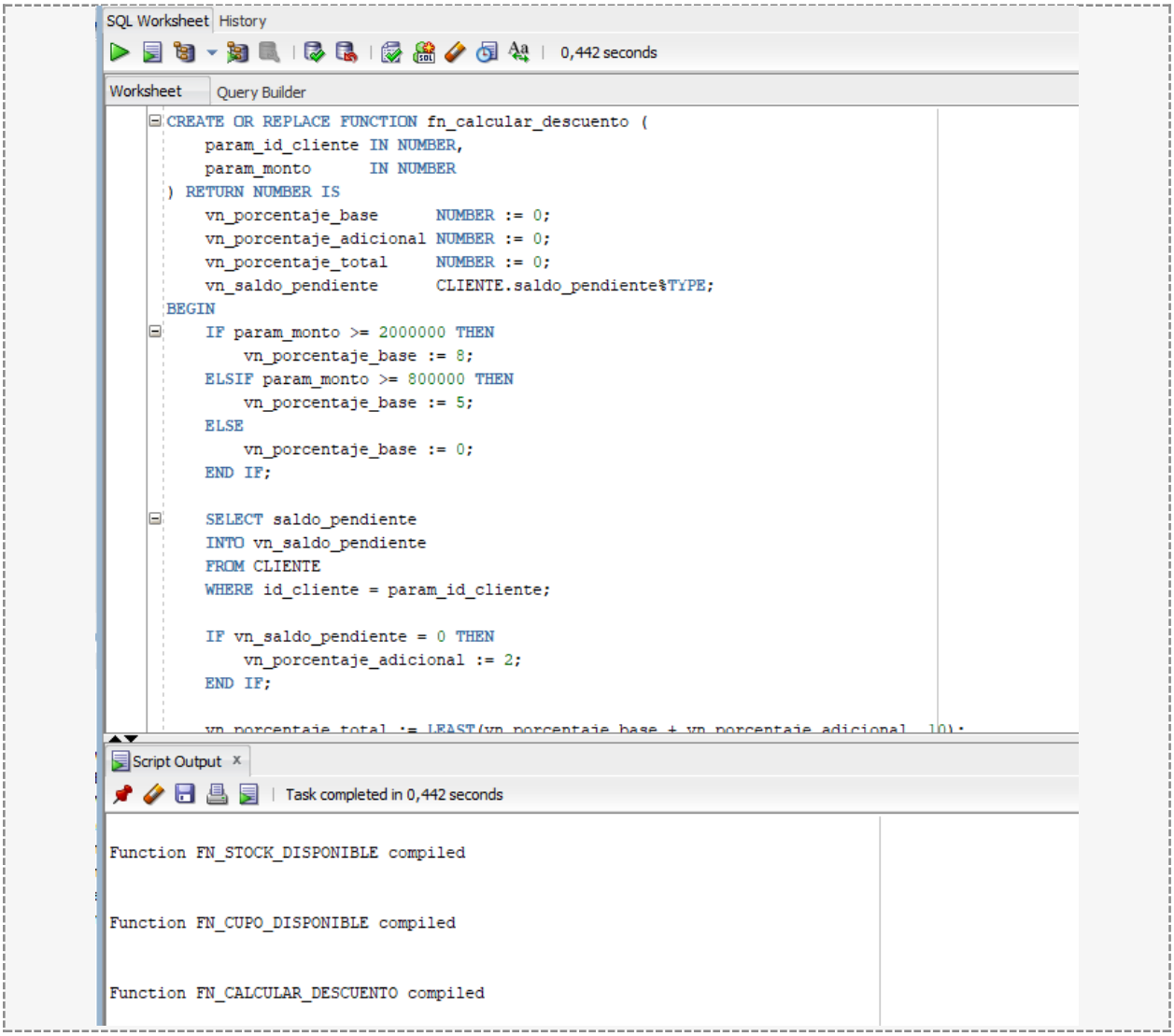
Explicación de la solución

Calcula el porcentaje base según el monto, le suma el 2 % si el cliente no tiene saldo pendiente y limita la suma con LEAST(…, 10). Retorna monto × porcentaje / 100, es decir, el valor en pesos y no el porcentaje, como exige el enunciado. Con las reglas actuales el máximo posible (8 + 2) coincide con el tope, por lo que LEAST protege ante cambios futuros de las tarifas. Un cliente inexistente devuelve 0.

Evidencias de ejecución

Captura 2.3 — FN_CALCULAR_DESCUENTO: casos de frontera y descuento adicional

Ejecutar: evidencias_pruebas.sql › E-2.3
Debe verse: La columna descuento igual a la columna esperado en las 9 filas.



2.4 Invocación desde SQL — 1 punto

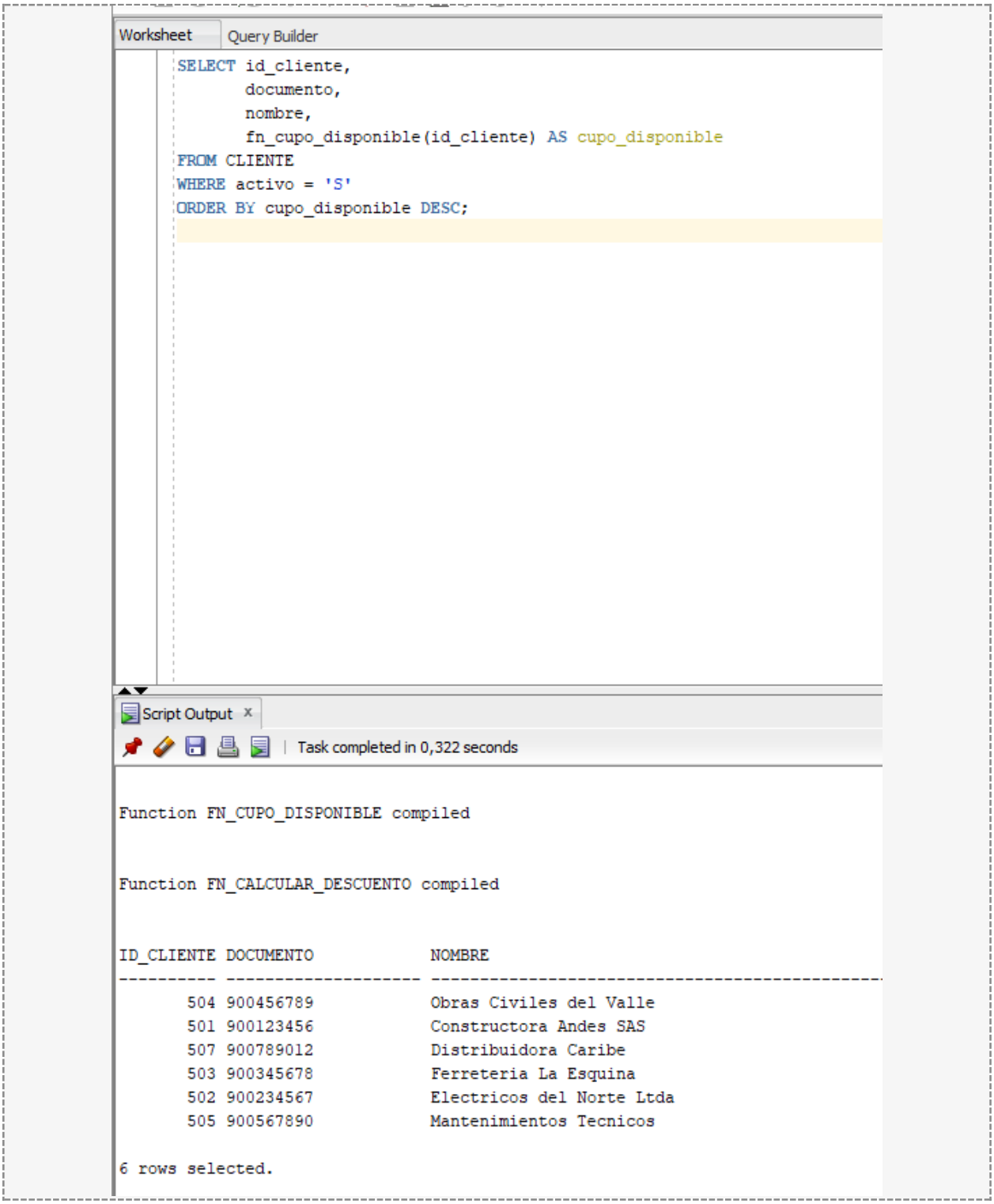
Enunciado: *Consulta de clientes activos con su cupo disponible (función 2.2), de mayor a menor; explicar por qué una función puede invocarse desde SQL y qué restricción cumple.*

Explicación de la solución

La consulta lista los clientes activos con fn_cupo_disponible y los ordena de mayor a menor cupo. Una función almacenada puede invocarse desde SQL porque es un objeto compilado del esquema que devuelve un tipo SQL y recibe parámetros IN; la restricción es de pureza: invocada desde una consulta no puede modificar el estado de la base de datos (no puede ejecutar INSERT, UPDATE, DELETE, DDL ni COMMIT; Oracle lo rechaza en ejecución con ORA-14551). Costo: la función se ejecuta una vez por fila, con un cambio de contexto PL/SQL–SQL en cada una.

Evidencias de ejecución

Captura 2.4 — Clientes activos ordenados por cupo disponible



Parte 3 — Cursores (10 puntos)

3.1 Cursor explícito — 2 puntos

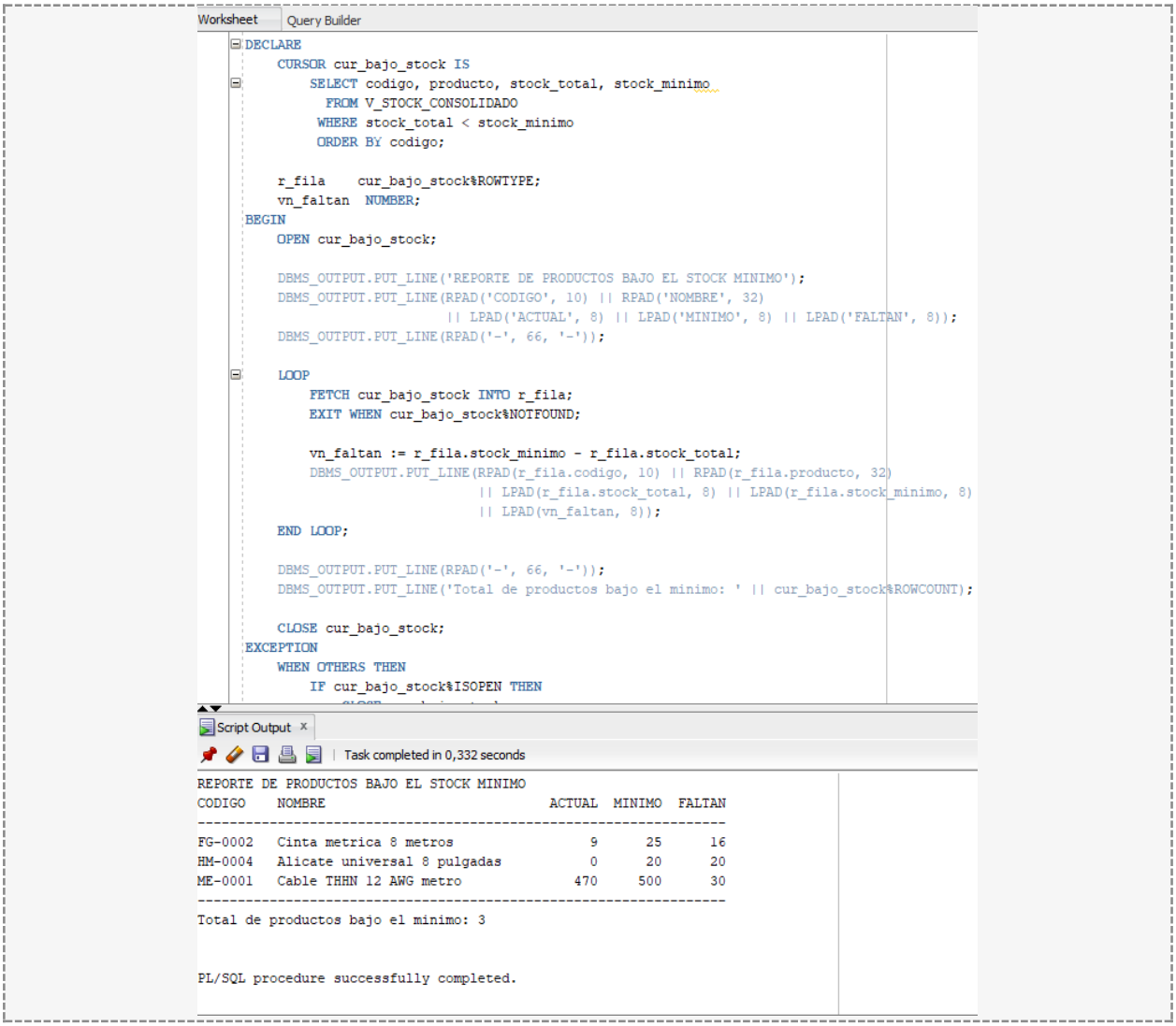
Enunciado: *Reporte de productos con stock consolidado bajo el mínimo, con OPEN, FETCH y CLOSE explícitos y el total de productos.*

Explicación de la solución

Cursor explícito sobre V_STOCK_CONSOLIDADO, que ya suma las existencias de todas las bodegas con un LEFT JOIN: así un producto sin ninguna existencia aparece con stock 0 en lugar de desaparecer del reporte. El filtro stock_total < stock_minimo está en el WHERE del cursor, por lo que PL/SQL solo recibe las filas a reportar. Usa OPEN, FETCH y CLOSE explícitos, con la prueba %NOTFOUND inmediatamente después del FETCH (si se evaluara al final, la última fila se imprimiría dos veces), y toma el total con %ROWCOUNT antes de cerrar el cursor.

Evidencias de ejecución

Captura 3.1 — Reporte de productos bajo el stock mínimo



Worksheet | Query Builder

```
DECLARE
    CURSOR cur_bajo_stock IS
        SELECT codigo, producto, stock_total, stock_minimo
          FROM V_STOCK_CONSOLIDADO
         WHERE stock_total < stock_minimo
         ORDER BY codigo;

    r_fila    cur_bajo_stock%ROWTYPE;
    vn_faltan  NUMBER;
BEGIN
    OPEN cur_bajo_stock;

    DBMS_OUTPUT.PUT_LINE('REPORTE DE PRODUCTOS BAJO EL STOCK MINIMO');
    DBMS_OUTPUT.PUT_LINE(RPAD('CODIGO', 10) || RPAD('NOMBRE', 32)
                         || LPAD('ACTUAL', 8) || LPAD('MINIMO', 8) || LPAD('FALTAN', 8));
    DBMS_OUTPUT.PUT_LINE(RPAD('-', 66, '-'));

    LOOP
        FETCH cur_bajo_stock INTO r_fila;
        EXIT WHEN cur_bajo_stock%NOTFOUND;

        vn_faltan := r_fila.stock_minimo - r_fila.stock_total;
        DBMS_OUTPUT.PUT_LINE(RPAD(r_fila.codigo, 10) || RPAD(r_fila.producto, 32)
                             || LPAD(r_fila.stock_total, 8) || LPAD(r_fila.stock_minimo, 8)
                             || LPAD(vn_faltan, 8));
    END LOOP;

    DBMS_OUTPUT.PUT_LINE(RPAD('-', 66, '-'));
    DBMS_OUTPUT.PUT_LINE('Total de productos bajo el minimo: ' || cur_bajo_stock%ROWCOUNT);

    CLOSE cur_bajo_stock;
EXCEPTION
    WHEN OTHERS THEN
        IF cur_bajo_stock%ISOPEN THEN
```

Script Output | Task completed in 0,332 seconds

```
REPORTE DE PRODUCTOS BAJO EL STOCK MINIMO
CODIGO    NOMBRE                           ACTUAL  MINIMO  FALTAN
------------------------------------------------------------------
FG-0002   Cinta metrica 8 metros                9      25      16
HM-0004   Alicate universal 8 pulgadas          0      20      20
ME-0001   Cable THHN 12 AWG metro             470     500      30
------------------------------------------------------------------
Total de productos bajo el minimo: 3


PL/SQL procedure successfully completed.
```

3.2 Cursor parametrizado — 2 puntos

Enunciado: *Pedidos confirmados de un cliente en un rango de fechas, con total y número de líneas; prueba con el cliente 501 en 2026.*

Explicación de la solución

El cursor declara sus tres parámetros (cliente, desde, hasta), por lo que se reutiliza con otros valores sin depender de variables del bloque. El rango es semiabierto (fecha ≥ desde y fecha < hasta + 1) porque fecha_pedido incluye hora; con BETWEEN se perderían los pedidos del último día. El LEFT JOIN con COUNT(d.linea) obtiene el número de líneas en la misma consulta y muestra con 0 un pedido confirmado sin líneas.

Evidencias de ejecución

Captura 3.2 — Pedidos confirmados del cliente 501 en 2026 (y reutilización con el 507)

```
DECLARE
    CURSOR cur_pedidos (param_cliente NUMBER, param_desde DATE, param_hasta DATE) IS
        SELECT p.id_pedido, p.fecha_pedido, p.total, COUNT(d.linea) AS lineas
          FROM PEDIDO p
          LEFT JOIN DETALLE_PEDIDO d ON d.id_pedido = p.id_pedido
         WHERE p.id_cliente   = param_cliente
           AND p.estado       = 'CONFIRMADO'
           AND p.fecha_pedido >= param_desde
           AND p.fecha_pedido <  param_hasta + 1
         GROUP BY p.id_pedido, p.fecha_pedido, p.total
         ORDER BY p.fecha_pedido, p.id_pedido;

    PROCEDURE sp_mostrar_pedidos (param_cli NUMBER, param_ini DATE, param_fin DATE) IS
        vn_pedidos  PLS_INTEGER := 0;
        vn_suma     NUMBER := 0;
    BEGIN
        DBMS_OUTPUT.PUT_LINE('Cliente ' || param_cli || ' entre ' || TO_CHAR(param_ini, 'YYYY-MM-DD')
                             || ' y ' || TO_CHAR(param_fin, 'YYYY-MM-DD'));
        FOR r_pedido IN cur_pedidos(param_cli, param_ini, param_fin) LOOP
            vn_pedidos    := vn_pedidos + 1;
            vn_suma := vn_suma + r_pedido.total;
            DBMS_OUTPUT.PUT_LINE('  Pedido ' || r_pedido.id_pedido || '  ' || TO_CHAR(r_pedido.fecha_pedido, 'YYYY-MM-DD')
                                 || '  total=' || r_pedido.total || '  lineas=' || r_pedido.lineas);
        END LOOP;
        IF vn_pedidos = 0 THEN
            DBMS_OUTPUT.PUT_LINE('  (sin pedidos confirmados en el rango)');
        ELSE
            DBMS_OUTPUT.PUT_LINE('  Pedidos: ' || vn_pedidos || '  suma de totales: ' || vn_suma);
        END IF;
    END sp_mostrar_pedidos;
BEGIN
    sp_mostrar_pedidos(501, DATE '2026-01-01', DATE '2026-12-31');
    sp_mostrar_pedidos(507, DATE '2026-01-01', DATE '2026-12-31');  -- reutiliza el cursor
END;
```

Script Output

Task completed in 0,424 seconds

```
Cliente 501 entre 2026-01-01 y 2026-12-31
  Pedido 1  2026-06-10  total=1286500  lineas=3
  Pedido 2  2026-07-02  total=641000  lineas=2
  Pedidos: 2  suma de totales: 1927500
Cliente 507 entre 2026-01-01 y 2026-12-31
  (sin pedidos confirmados en el rango)


PL/SQL procedure successfully completed.
```

3.3 Cursor FOR UPDATE — 3 puntos

Enunciado: *Ajusta el precio de los productos de una categoría con WHERE CURRENT OF, sin tocar inactivos y respetando costo + margen mínimo; explica el bloqueo.*

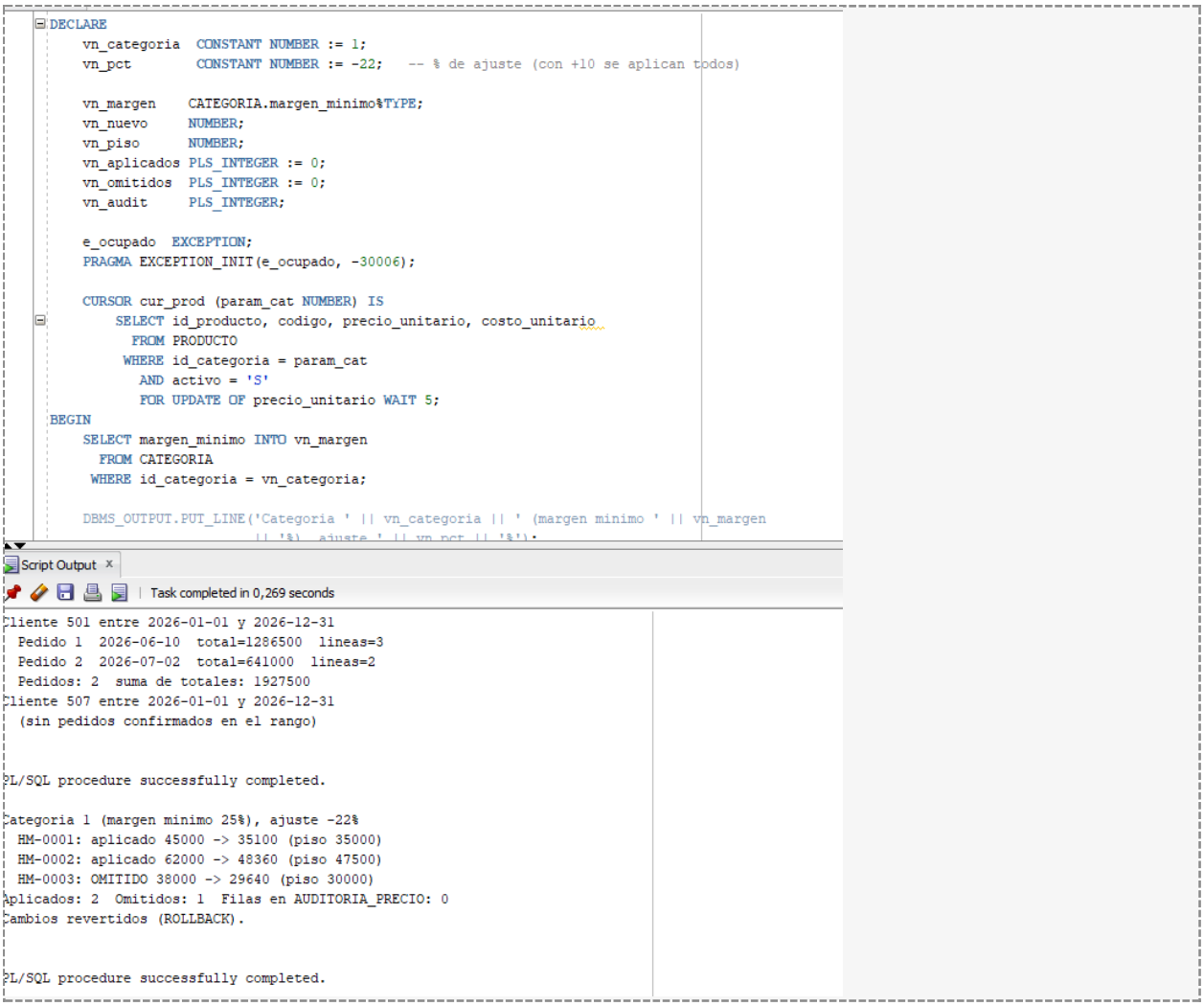
Explicación de la solución

El cursor FOR UPDATE filtra en el WHERE los productos activos de la categoría, de modo que los inactivos ni se leen ni se bloquean. Para cada fila calcula el nuevo precio, lo compara con costo × (1 + margen/100) y actualiza con WHERE CURRENT OF solo si lo respeta; si no, lo omite e informa. FOR UPDATE bloquea en modo exclusivo todas las filas del cursor al abrirlo, y el bloqueo dura hasta COMMIT o ROLLBACK (no hasta CLOSE); WAIT 5 evita esperar indefinidamente.

La protección contra la pérdida de actualización la da el bloqueo; WHERE CURRENT OF aporta acceso por ROWID sin recorrer el índice de la llave primaria y garantiza que se modifica exactamente la fila posicionada. Los UPDATE disparan TRG_PRODUCTO_BIU_MARGEN y TRG_PRODUCTO_AU_PRECIO.

Evidencias de ejecución

Captura 3.3-a — Ajuste de precios de la categoría 1 (-22 %)



```
DECLARE
    vn_categoria  CONSTANT NUMBER := 1;
    vn_pct        CONSTANT NUMBER := -22;   -- % de ajuste (con +10 se aplican todos)

    vn_margen    CATEGORIA.margen_minimo%TYPE;
    vn_nuevo     NUMBER;
    vn_piso      NUMBER;
    vn_aplicados PLS_INTEGER := 0;
    vn_omitidos  PLS_INTEGER := 0;
    vn_audit     PLS_INTEGER;

    e_ocupado  EXCEPTION;
    PRAGMA EXCEPTION_INIT(e_ocupado, -30006);

    CURSOR cur_prod (param_cat NUMBER) IS
        SELECT id_producto, codigo, precio_unitario, costo_unitario
          FROM PRODUCTO
         WHERE id_categoria = param_cat
           AND activo = 'S'
           FOR UPDATE OF precio_unitario WAIT 5;
BEGIN
    SELECT margen_minimo INTO vn_margen
      FROM CATEGORIA
     WHERE id_categoria = vn_categoria;

    DBMS_OUTPUT.PUT_LINE('Categoria ' || vn_categoria || ' (margen minimo ' || vn_margen
                         || '%), ajuste ' || vn_pct || '%');
```

```
Script Output
Task completed in 0,269 seconds

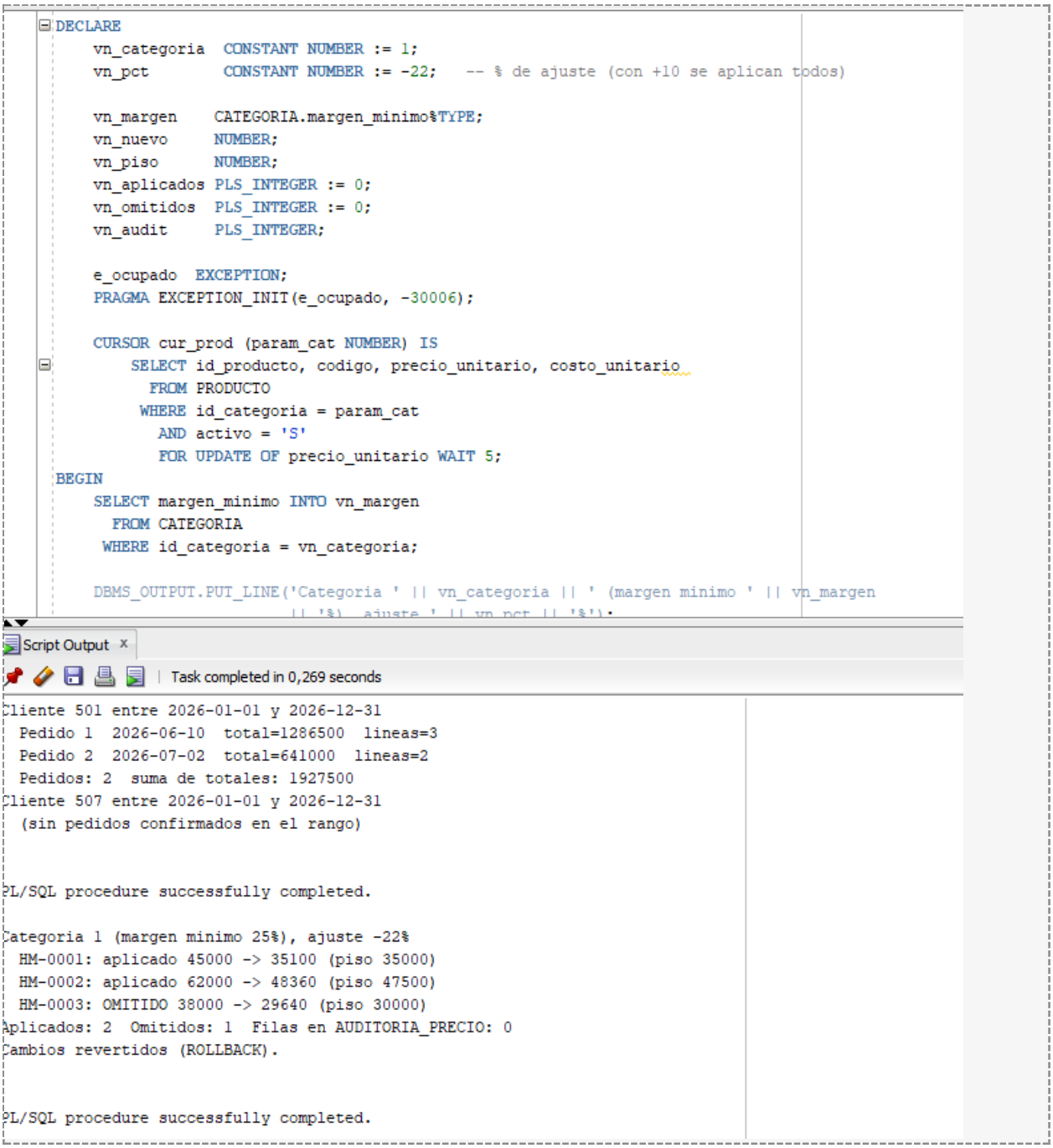
Cliente 501 entre 2026-01-01 y 2026-12-31
  Pedido 1  2026-06-10  total=1286500  lineas=3
  Pedido 2  2026-07-02  total=641000  lineas=2
  Pedidos: 2  suma de totales: 1927500
Cliente 507 entre 2026-01-01 y 2026-12-31
  (sin pedidos confirmados en el rango)


PL/SQL procedure successfully completed.

Categoria 1 (margen minimo 25%), ajuste -22%
  HM-0001: aplicado 45000 -> 35100 (piso 35000)
  HM-0002: aplicado 62000 -> 48360 (piso 47500)
  HM-0003: OMITIDO 38000 -> 29640 (piso 30000)
Aplicados: 2  Omitidos: 1  Filas en AUDITORIA_PRECIO: 0
Cambios revertidos (ROLLBACK).


PL/SQL procedure successfully completed.
```

Captura 3.3-b — FOR UPDATE frente a otra sesión *(opcional)*



```
DECLARE
    vn_categoria  CONSTANT NUMBER := 1;
    vn_pct        CONSTANT NUMBER := -22;   -- % de ajuste (con +10 se aplican todos)

    vn_margen    CATEGORIA.margen_minimo%TYPE;
    vn_nuevo     NUMBER;
    vn_piso      NUMBER;
    vn_aplicados PLS_INTEGER := 0;
    vn_omitidos  PLS_INTEGER := 0;
    vn_audit     PLS_INTEGER;

    e_ocupado  EXCEPTION;
    PRAGMA EXCEPTION_INIT(e_ocupado, -30006);

    CURSOR cur_prod (param_cat NUMBER) IS
        SELECT id_producto, codigo, precio_unitario, costo_unitario
          FROM PRODUCTO
         WHERE id_categoria = param_cat
           AND activo = 'S'
           FOR UPDATE OF precio_unitario WAIT 5;
BEGIN
    SELECT margen_minimo INTO vn_margen
      FROM CATEGORIA
     WHERE id_categoria = vn_categoria;

    DBMS_OUTPUT.PUT_LINE('Categoria ' || vn_categoria || ' (margen minimo ' || vn_margen
```

Script Output

Task completed in 0,269 seconds

```
Cliente 501 entre 2026-01-01 y 2026-12-31
  Pedido 1  2026-06-10  total=1286500  lineas=3
  Pedido 2  2026-07-02  total=641000  lineas=2
  Pedidos: 2  suma de totales: 1927500
Cliente 507 entre 2026-01-01 y 2026-12-31
  (sin pedidos confirmados en el rango)


PL/SQL procedure successfully completed.

Categoria 1 (margen minimo 25%), ajuste -22%
  HM-0001: aplicado 45000 -> 35100 (piso 35000)
  HM-0002: aplicado 62000 -> 48360 (piso 47500)
  HM-0003: OMITIDO 38000 -> 29640 (piso 30000)
Aplicados: 2  Omitidos: 1  Filas en AUDITORIA_PRECIO: 0
Cambios revertidos (ROLLBACK).


PL/SQL procedure successfully completed.
```

3.4 Cursor de referencia: SP_LISTAR_MOVIMIENTOS — 2 puntos

Enunciado: *Procedimiento que devuelve un SYS_REFCURSOR con los movimientos de un producto ordenados por fecha, y un bloque que lo consume.*

Explicación de la solución

SP_LISTAR_MOVIMIENTOS abre un SYS_REFCURSOR (cursor débil) sobre los movimientos del producto ordenados por fecha e id, y lo entrega por el parámetro OUT sin leerlo ni cerrarlo: recorrerlo y cerrarlo es responsabilidad de quien lo recibe. El bloque consumidor hace FETCH hasta %NOTFOUND, cuenta con %ROWCOUNT y cierra el cursor (también si ocurre un error). El ORDER BY es necesario porque Oracle no garantiza ningún orden sin él.

Evidencias de ejecución

Captura 3.4 — Creación del procedimiento y consumo del cursor devuelto

Worksheet | Query Builder

```
CREATE OR REPLACE PROCEDURE SP_LISTAR_MOVIMIENTOS (
    param_id_producto IN  NUMBER,
    param_cursor      OUT SYS_REFCURSOR
) IS
BEGIN
    OPEN param_cursor FOR
        SELECT id_movimiento, id_producto, id_bodega, tipo, cantidad,
               fecha_mov, id_pedido, usuario_bd, observacion
          FROM MOVIMIENTO_INVENTARIO
         WHERE id_producto = param_id_producto
         ORDER BY fecha_mov, id_movimiento;
END SP_LISTAR_MOVIMIENTOS;
/

-- Bloque consumidor (esperado: producto 1001 -> 4 movimientos; 1015 -> ninguno):
DECLARE
    PROCEDURE sp_consumir_movimientos (param_id NUMBER) IS
        cur_mov  SYS_REFCURSOR;
        r_mov    MOVIMIENTO_INVENTARIO%ROWTYPE;
    BEGIN
        SP_LISTAR_MOVIMIENTOS(param_id, cur_mov);
        DBMS_OUTPUT.PUT_LINE('Movimientos del producto ' || param_id || ':');
        BEGIN
            LOOP
                FETCH cur_mov INTO r_mov;
                EXIT WHEN cur_mov%NOTFOUND;
                DBMS_OUTPUT.PUT_LINE('  #' || r_mov.id_movimiento || '  '
```

Script Output | Task completed in 0,989 seconds

```
Procedure SP_LISTAR_MOVIMIENTOS compiled

Movimientos del producto 1001:
  #1  2026-05-02  ENTRADA  cant=150  bodega=1  Compra a proveedor
  #6  2026-06-10  SALIDA   cant=5  bodega=1  Despacho de pedido
  #8  2026-06-20  ENTRADA  cant=40  bodega=2  Traslado entre bodegas
  #20  2026-08-25  AJUSTE   cant=25  bodega=1  Conteo fisico
  #21  2026-09-30  ENTRADA  cant=10  bodega=1  Prueba entrada
  #22  2026-09-30  ENTRADA  cant=10  bodega=1  Prueba entrada
  #23  2026-09-30  SALIDA   cant=5  bodega=1  Prueba salida
  #25  2026-09-30  AJUSTE   cant=50  bodega=1  Conteo fisico
  Total: 8
Movimientos del producto 1015:
  Total: 0


PL/SQL procedure successfully completed.
```

3.5 BULK COLLECT y FORALL — 1 punto

Enunciado: *Carga con BULK COLLECT los productos de una categoría y actualiza su stock mínimo con FORALL; explica el costo que elimina.*

Explicación de la solución

BULK COLLECT carga en colecciones, en un solo paso, los ids de los productos de la categoría y su nuevo stock mínimo calculado en el mismo SELECT; FORALL envía después el UPDATE una sola vez, con los arreglos como variables enlazadas. Elimina el cambio de contexto entre el motor PL/SQL y el motor SQL que ocurre por cada fila en un ciclo con UPDATE. Con las pocas filas del taller no se mide; empieza a notarse desde cientos de filas y es determinante en miles. Con volúmenes grandes se procesa por lotes con LIMIT para acotar la memoria.

Evidencias de ejecución

Captura 3.5 — Actualización masiva del stock mínimo de la categoría 1

Worksheet | Query Builder

```
DECLARE
    vn_categoria  CONSTANT NUMBER := 1;
    vn_factor     CONSTANT NUMBER := 1.10;

    TYPE t_ids IS TABLE OF PRODUCTO.id_producto%TYPE;
    TYPE t_min IS TABLE OF PRODUCTO.stock_minimo%TYPE;
    vt_ids  t_ids;
    vt_min  t_min;
BEGIN
    SELECT id_producto, CEIL(stock_minimo * vn_factor)
      BULK COLLECT INTO vt_ids, vt_min
      FROM PRODUCTO
     WHERE id_categoria = vn_categoria
     ORDER BY id_producto;

    DBMS_OUTPUT.PUT_LINE('Productos cargados en la coleccion: ' || vt_ids.COUNT);

    IF vt_ids.COUNT > 0 THEN
        FORALL vn_i IN 1 .. vt_ids.COUNT
            UPDATE PRODUCTO
               SET stock_minimo = vt_min(vn_i)
             WHERE id_producto = vt_ids(vn_i);

        DBMS_OUTPUT.PUT_LINE('Filas actualizadas por FORALL: ' || SQL%ROWCOUNT);
        FOR vn_i IN 1 .. vt_ids.COUNT LOOP
            DBMS_OUTPUT.PUT_LINE('  producto ' || vt_ids(vn_i) || ' -> stock minimo '
                                 || vt_min(vn_i) || '  (filas: ' || SQL%BULK_ROWCOUNT(vn_i) || ')');
```

Script Output | Task completed in 0,353 seconds

```
Productos cargados en la coleccion: 4
Filas actualizadas por FORALL: 4
  producto 1001 -> stock minimo 22  (filas: 1)
  producto 1002 -> stock minimo 17  (filas: 1)
  producto 1003 -> stock minimo 28  (filas: 1)
  producto 1015 -> stock minimo 22  (filas: 1)
Cambios revertidos (ROLLBACK).


PL/SQL procedure successfully completed.
```

Parte 4 — Manejo de excepciones (9 puntos)

4.1 FN_PRECIO_PRODUCTO — 2 puntos

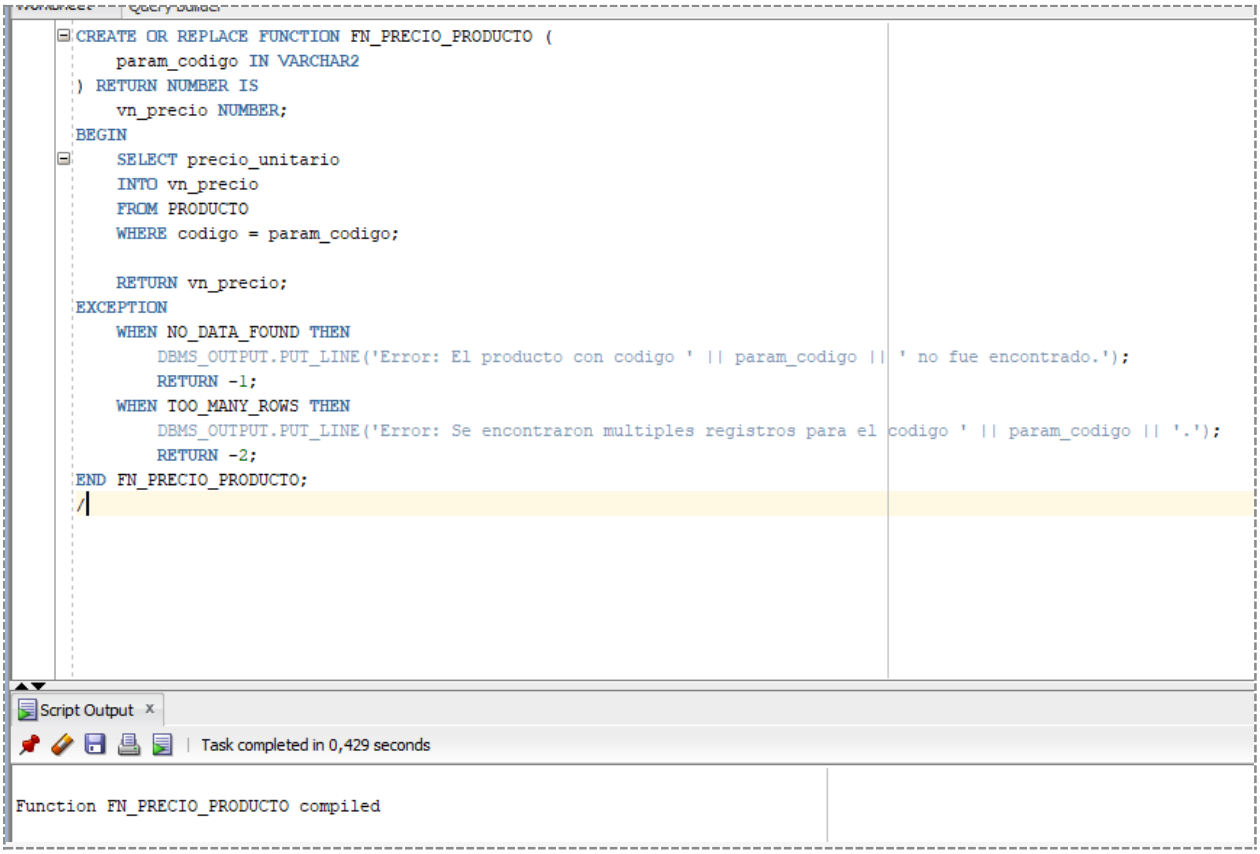
Enunciado: *Retorna el precio de un producto por su código; NO_DATA_FOUND → -1 y TOO_MANY_ROWS → -2, con mensaje descriptivo.*

Explicación de la solución

La función busca el producto por su código y maneja explícitamente NO_DATA_FOUND (imprime que no existe y retorna -1) y TOO_MANY_ROWS (retorna -2). Sobre PRODUCTO, TOO_MANY_ROWS no puede ocurrir mientras UK_PRODUCTO_CODIGO esté habilitada: solo sería posible si un administrador deshabilitara esa restricción (o la dejara sin validar) y se cargaran códigos repetidos; por eso no se puede reproducir con el esquema tal como está.

Evidencias de ejecución

Captura 4.1 — FN_PRECIO_PRODUCTO con código existente e inexistente



4.2 SP_VALIDAR_DESPACHO — 2 puntos

Enunciado: *Verifica si puede despacharse una cantidad desde una bodega, con excepción propia levantada con RAISE y capturada en EXCEPTION.*

Explicación de la solución

Obtiene el disponible con SUM/NVL (cero si la combinación no existe), declara la excepción e_stock_insuficiente y la levanta con RAISE cuando la cantidad supera el disponible; la captura en su bloque EXCEPTION. Nombrar la condición separa el camino normal del tratamiento del error de negocio, evita IF anidados y deja un único lugar donde registrar o propagar ese error.

Evidencias de ejecución

Captura 4.2 — Despacho válido, excedido y sobre producto sin existencia

Ejecutar: evidencias_pruebas.sql › E-4.2
Debe verse: 5 unidades: validación exitosa; 999999: error de negocio con disponible 120; producto 1015: disponible 0.

```
CREATE OR REPLACE PROCEDURE SP_VALIDAR_DESPACHO (
    param_id_producto IN NUMBER,
    param_id_bodega   IN NUMBER,
    param_cantidad    IN NUMBER
) IS
    vn_disponible NUMBER;
    e_stock_insuficiente EXCEPTION;
BEGIN
    -- Obtenemos el disponible (usamos NVL por si no existe la relacion en la bodega)
    SELECT NVL(SUM(cantidad), 0)
    INTO vn_disponible
    FROM EXISTENCIA
    WHERE id_producto = param_id_producto
      AND id_bodega = param_id_bodega;

    IF param_cantidad > vn_disponible THEN
        RAISE e_stock_insuficiente;
    END IF;

    DBMS_OUTPUT.PUT_LINE('Validacion exitosa: Hay stock suficiente para el despacho.');
EXCEPTION
    WHEN e_stock_insuficiente THEN
        DBMS_OUTPUT.PUT_LINE('Error de Negocio: La cantidad requerida (' || param_cantidad ||
                             ') supera el inventario disponible (' || vn_disponible || ').');
END SP_VALIDAR_DESPACHO;
/
```

```
Script Output
Task completed in 0,456 seconds

Function FN_PRECIO_PRODUCTO compiled


Procedure SP_VALIDAR_DESPACHO compiled
```

)

4.3 SP_ANULAR_PEDIDO — 2 puntos

Enunciado: *Anula un pedido con RAISE_APPLICATION_ERROR: -20010 no existe, -20011 ya anulado, -20012 más de 30 días.*

Explicación de la solución

Verifica en este orden: existencia (NO_DATA_FOUND → -20010), estado ya anulado (-20011) y antigüedad mayor de 30 días (-20012); si pasa, cambia el estado a ANULADO. RAISE solo levanta una excepción PL/SQL (nombrada o predefinida) y, si nadie la captura, el cliente recibe un error genérico (ORA-06510 para una definida por el usuario); RAISE_APPLICATION_ERROR levanta un error con código (-20000 a -20999) y mensaje propios que viajan hasta el cliente, que puede distinguirlos por el código.

Evidencias de ejecución

Captura 4.3 — Anulación exitosa y los tres errores propios

Worksheet | Query Builder

```
CREATE OR REPLACE PROCEDURE SP_ANULAR_PEDIDO (
    param_id_pedido IN NUMBER
) IS
    vv_estado       PEDIDO.estado%TYPE;
    vd_fecha_pedido PEDIDO.fecha_pedido%TYPE;
BEGIN
    SELECT estado, fecha_pedido
    INTO vv_estado, vd_fecha_pedido
    FROM PEDIDO
    WHERE id_pedido = param_id_pedido;

    IF vv_estado = 'ANULADO' THEN
        RAISE_APPLICATION_ERROR(-20011, 'El pedido ya esta anulado.');
    ELSIF (SYSDATE - vd_fecha_pedido) > 30 THEN
        RAISE_APPLICATION_ERROR(-20012, 'El pedido tiene mas de 30 dias y no puede anularse.');
    END IF;

    UPDATE PEDIDO
    SET estado = 'ANULADO'
    WHERE id_pedido = param_id_pedido;

    DBMS_OUTPUT.PUT_LINE('El pedido ' || param_id_pedido || ' ha sido anulado.');

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20010, 'El pedido especificado no existe.');
END SP_ANULAR_PEDIDO;
```

Script Output ×

Task completed in 1,068 seconds

```
Function FN_PRECIO_PRODUCTO compiled


Procedure SP_VALIDAR_DESPACHO compiled


Procedure SP_ANULAR_PEDIDO compiled
```

4.4 PRAGMA EXCEPTION_INIT — 2 puntos

Enunciado: *Inserta un producto con categoría inexistente y captura ORA-02291 asociado a una excepción con nombre.*

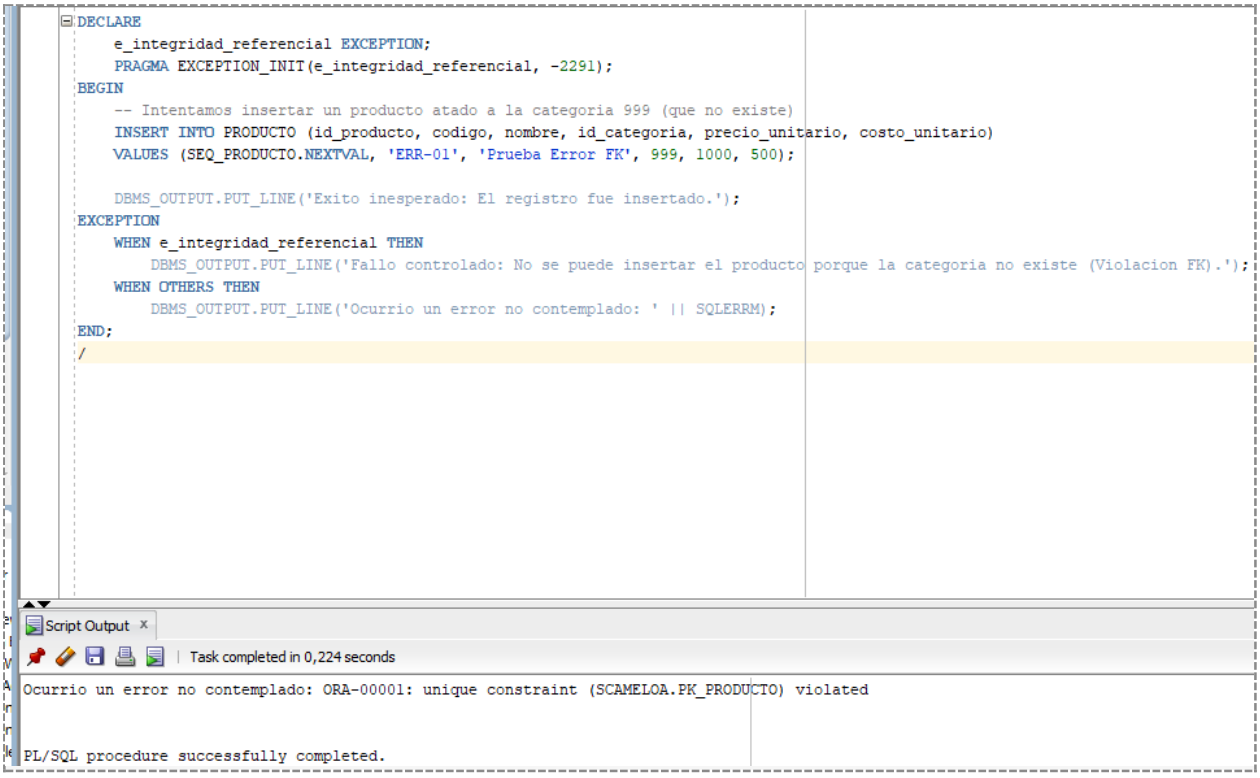
Explicación de la solución

El bloque inserta un producto con categoría 999 y asocia ORA-02291 (violación de llave foránea: no se encontró la llave padre) a la excepción e_integridad_referencial con PRAGMA EXCEPTION_INIT. Frente a WHEN OTHERS, la excepción nombrada reacciona solo a ese error y deja propagar cualquier otro; WHEN OTHERS atraparía también fallos inesperados (espacio, privilegios) y los enmascararía. Para que llegue ORA-02291, TRG_PRODUCTO_BIU_MARGEN deja pasar las categorías inexistentes y la secuencia debe estar sincronizada (de lo contrario se obtendría ORA-00001 antes).

Evidencias de ejecución

Captura 4.4 — Captura de ORA-02291 con la excepción nombrada

.



4.5 SP_REGISTRAR_ERROR — 1 punto

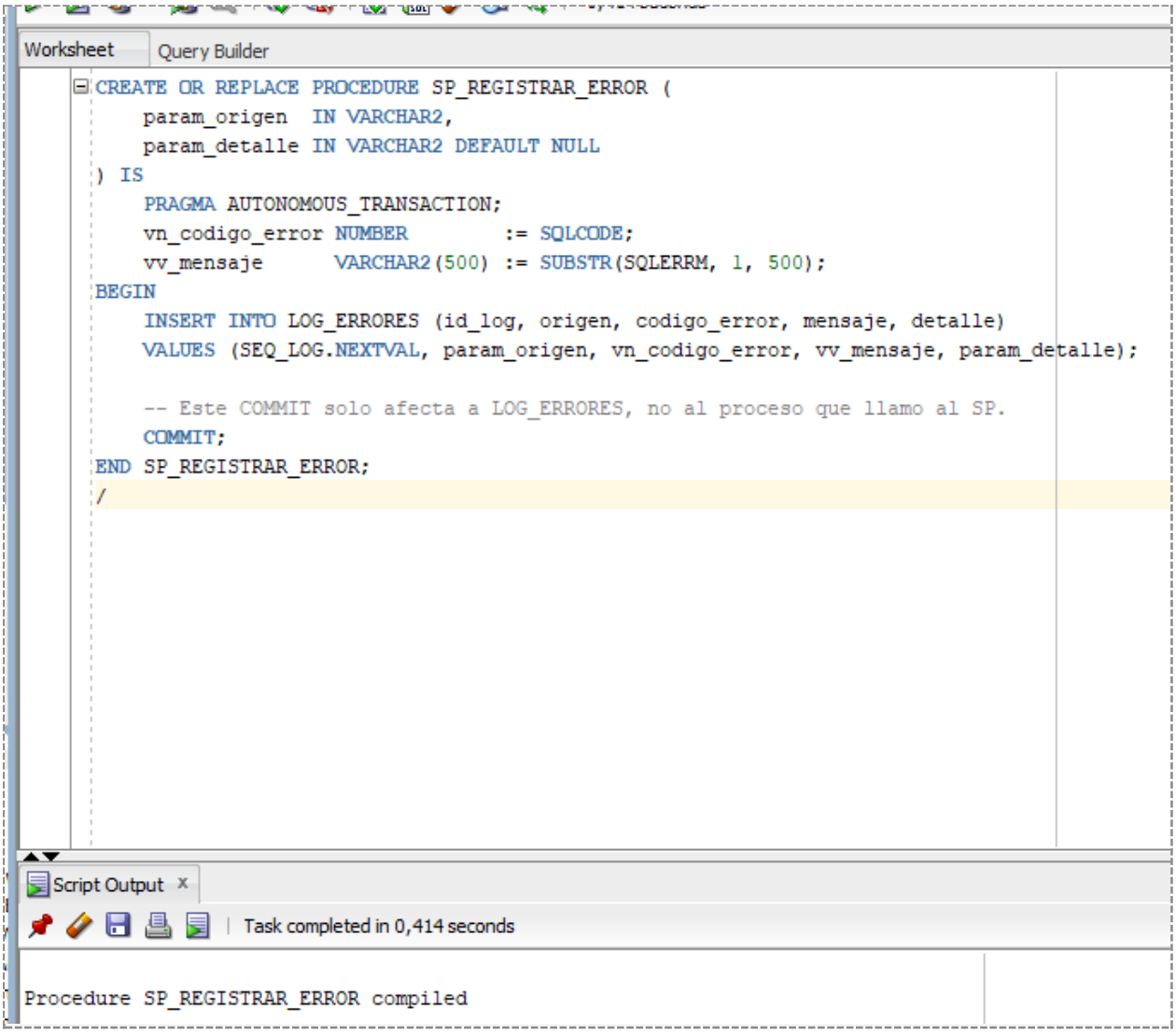
Enunciado: *Registra SQLCODE y SQLERRM actuales en LOG_ERRORES de forma que persista aunque la transacción principal termine en ROLLBACK.*

Explicación de la solución

Toma SQLCODE y SQLERRM al entrar, inserta en LOG_ERRORES con SEQ_LOG y confirma con COMMIT. PRAGMA AUTONOMOUS_TRANSACTION ejecuta el procedimiento en una transacción independiente: su COMMIT confirma solo el registro y no la transacción de negocio, y un ROLLBACK posterior del llamador no borra el log. La misma independencia puede ocultar información cuando el log queda confirmado aunque el llamador reintente y termine con éxito (el registro sugiere una falla que no persistió), o cuando el procedimiento autónomo necesita leer datos aún no confirmados del llamador, que no puede ver.

Evidencias de ejecución

Captura 4.5 — El registro persiste después del ROLLBACK de la transacción principal



Parte 5 — Disparadores (10 puntos)

5.1 TRG_CLIENTE_BI — 2 puntos

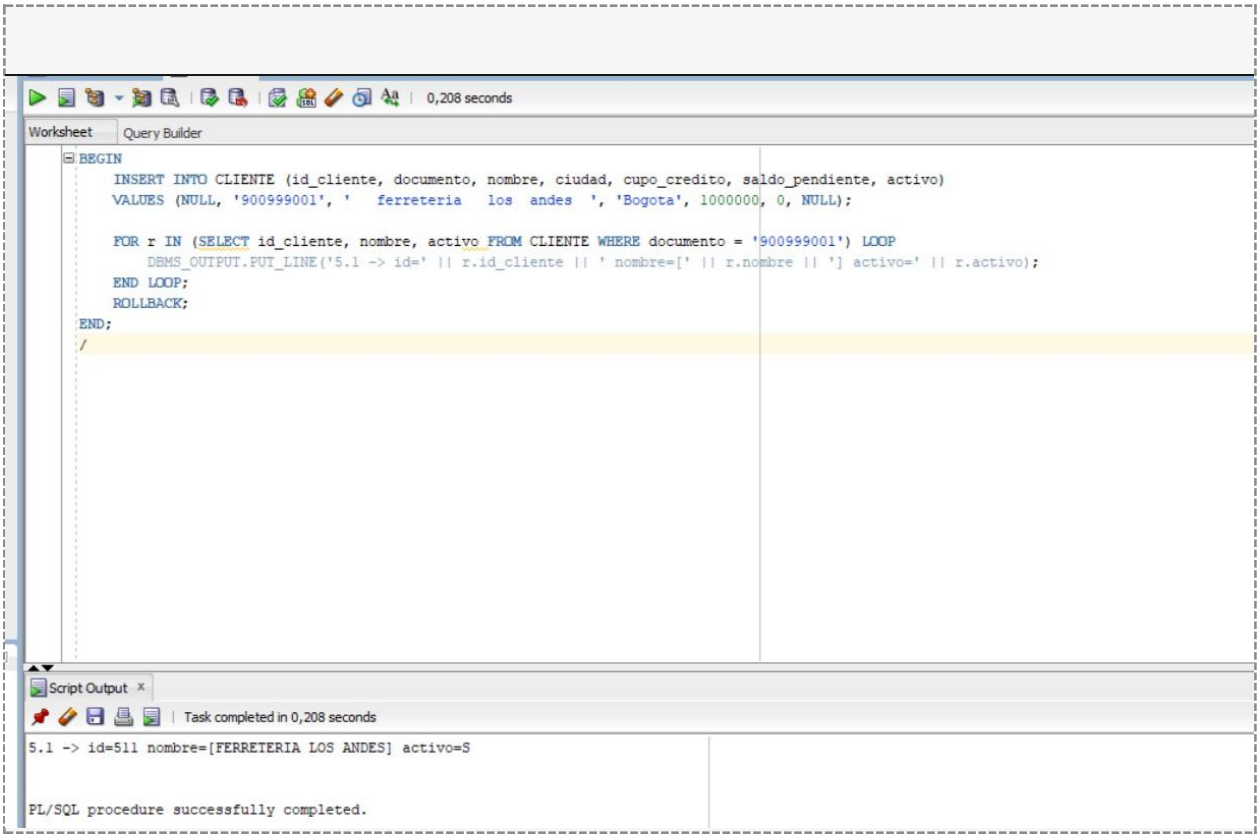
Enunciado: *BEFORE INSERT sobre CLIENTE: id desde la secuencia si llega nulo, nombre en mayúsculas sin espacios sobrantes y estado activo 'S' por defecto.*

Explicación de la solución

BEFORE INSERT de fila, porque solo ahí se puede modificar :NEW antes de que se validen las restricciones. Asigna el id desde SEQ_CLIENTE solo si llega nulo (respeta ids explícitos), normaliza el nombre con UPPER(TRIM(…)) colapsando los espacios internos repetidos y fuerza activo = 'S' cuando llega nulo (el DEFAULT de la columna no actúa si se envía NULL de forma explícita). Nota operacional: SEQ_CLIENTE arranca en 500 y el script 01 no la sincronizó con los clientes 501–507, por lo que debe avanzarse antes de usarla.

Evidencias de ejecución

Captura 5.1 — Inserción de un cliente con id nulo, nombre sucio y activo nulo



5.2 TRG_PRODUCTO_AU_PRECIO — 2 puntos

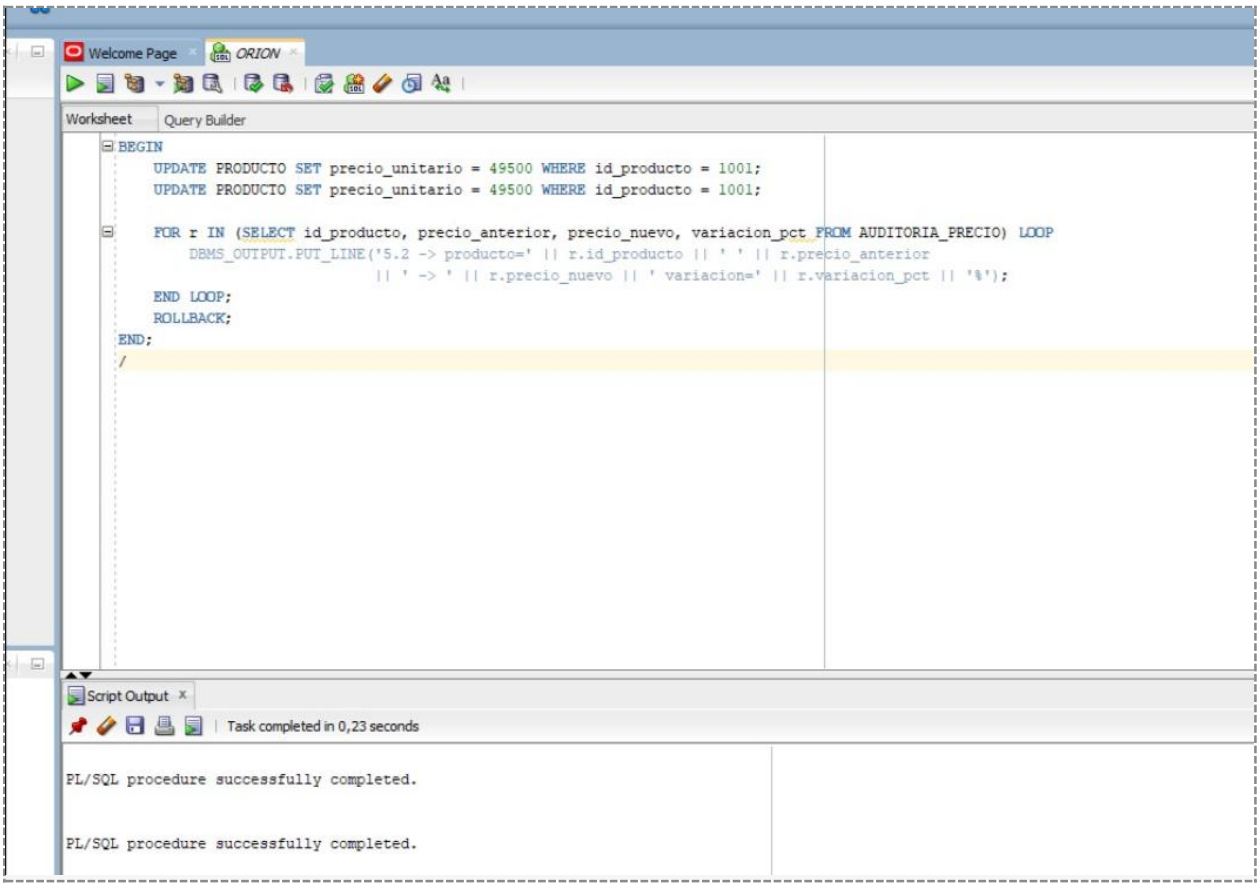
Enunciado: *AFTER UPDATE: audita cada cambio de precio con la variación porcentual, disparándose solo si el precio cambia (cláusula WHEN).*

Explicación de la solución

AFTER UPDATE OF precio_unitario con la cláusula WHEN (NEW.precio_unitario <> OLD.precio_unitario). WHEN lo evalúa el motor antes de ejecutar el cuerpo: si el precio no cambió, el trigger no se ejecuta, mientras que un IF dentro del cuerpo ejecutaría el trigger en cada fila actualizada y recién ahí descartaría. UPDATE OF solo filtra por columnas mencionadas, no por valor; por eso se necesita el WHEN. Registra precio anterior, nuevo y la variación porcentual redondeada a 2 decimales.

Evidencias de ejecución

Captura 5.2 — Cambio de precio auditado y UPDATE con el mismo valor sin auditoría



5.3 TRG_PRODUCTO_BIU_MARGEN — 2 puntos

Enunciado: *BEFORE INSERT OR UPDATE: rechaza con -20020 un producto cuyo precio no alcance costo × (1 + margen/100) de su categoría.*

Explicación de la solución

BEFORE INSERT OR UPDATE OF (precio, costo, categoría): consulta el margen mínimo de la categoría y rechaza con -20020 si precio < costo × (1 + margen/100). Si la categoría no existe no levanta un error propio: deja que la llave foránea responda con ORA-02291, que es la causa real y la que necesita capturar el punto 4.4. Límite: si luego cambia el margen de una categoría, los productos existentes no se revalidan hasta que se modifiquen.

Evidencias de ejecución

Captura 5.3 — Precio bajo el margen, precio en el límite y categoría inexistente

```
DECLARE
    PROCEDURE intentar(p_desc VARCHAR2, p_sql VARCHAR2) IS
    BEGIN
        EXECUTE IMMEDIATE p_sql;
        DBMS_OUTPUT.PUT_LINE('5.3 ' || p_desc || ' -> OK');
    EXCEPTION
        WHEN OTHERS THEN
            DBMS_OUTPUT.PUT_LINE('5.3 ' || p_desc || ' -> ' || SQLCODE || ' ' || SQLERRM);
    END;
BEGIN
    intentar('precio bajo el margen',
             'UPDATE PRODUCTO SET precio_unitario = 30000 WHERE id_producto = 1001');
    intentar('precio en el limite',
             'UPDATE PRODUCTO SET precio_unitario = 35000 WHERE id_producto = 1001');
    intentar('categoria inexistente',
             'INSERT INTO PRODUCTO VALUES (9999, ''ZZ-9999'', ''Prueba'', 99, 1000, 500, 0, ''S'')');
    ROLLBACK;
END;
/
```

```
Script Output
Task completed in 0,248 seconds
5.3 precio bajo el margen -> -20020 ORA-20020: Precio 30000 inferior al minimo permitido 35000 (costo 28000 con margen minimo de 25% de la categoria 1)
5.3 precio en el limite -> OK
5.3 categoria inexistente -> -2291 ORA-02291: integrity constraint (SCAMELOA.FK_PRODUCTO_CATEGORIA) violated - parent key not found


PL/SQL procedure successfully completed.
```

5.4 TRG_DETALLE_AI_STOCK — 2 puntos

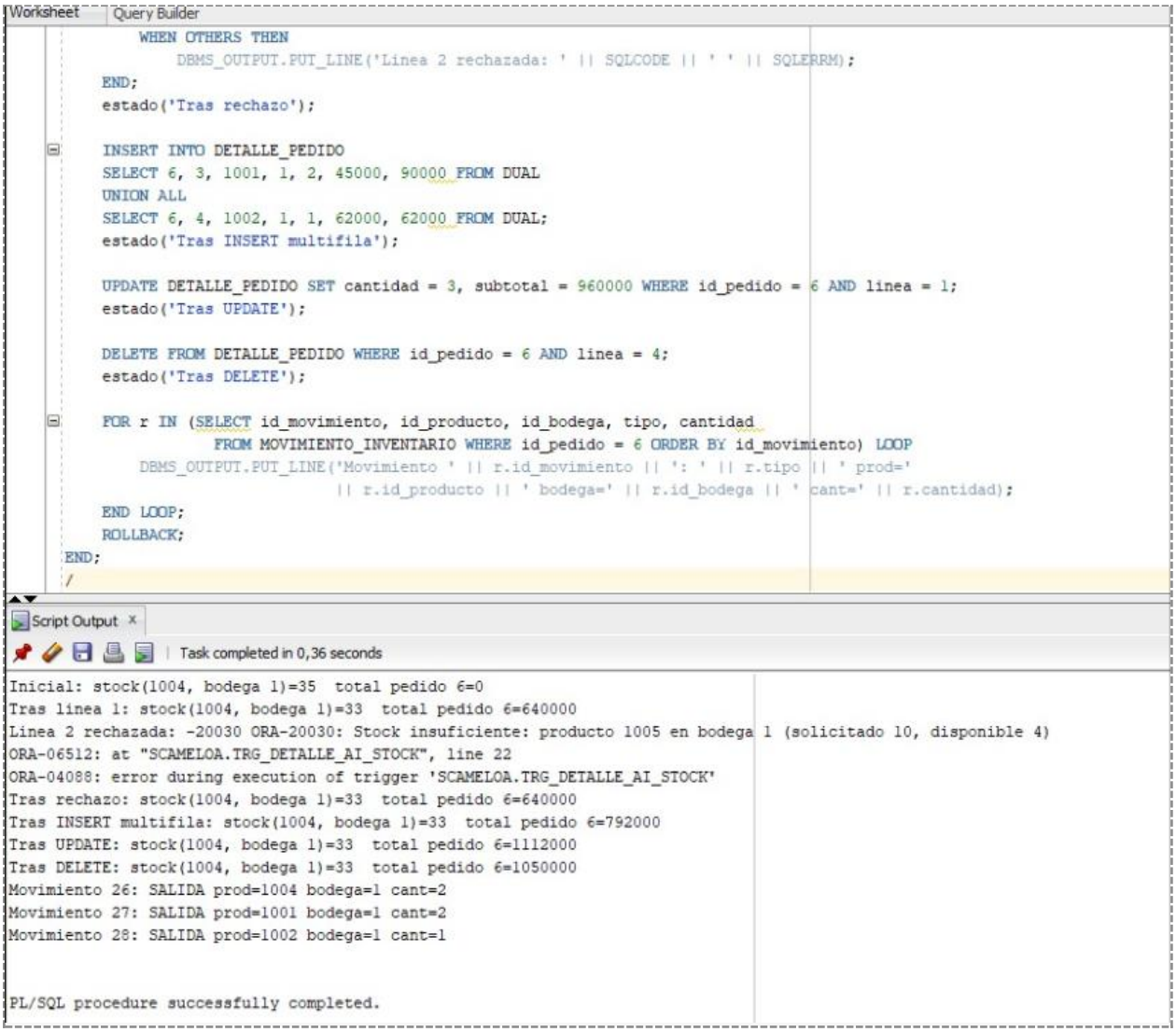
Enunciado: *AFTER INSERT sobre DETALLE_PEDIDO: descuenta el stock de la bodega y registra la SALIDA; sin stock suficiente aborta con -20030.*

Explicación de la solución

AFTER INSERT de fila: cuando se descuenta el stock, la línea ya superó las restricciones (PK, FK, CHECK de cantidad); en un BEFORE se actuaría sobre una fila aún no validada (una cantidad negativa, por ejemplo, sumaría stock antes de ser rechazada). El descuento y la verificación ocurren en un solo UPDATE … WHERE cantidad >= :NEW.cantidad: si no afecta filas (stock insuficiente o combinación inexistente) se aborta con -20030, sin ventana de concurrencia entre leer y descontar. Después inserta el movimiento de SALIDA con SEQ_MOVIMIENTO. Solo cubre INSERT: modificar o borrar líneas no devuelve stock.

Evidencias de ejecución

Captura 5.4 — Descuento de stock, movimiento de SALIDA y rechazo por stock insuficiente



5.5 TRG_DETALLE_TOTAL (COMPOUND TRIGGER) — 1 punto

Enunciado: *Recalcula el total de la cabecera tras insertar, actualizar o borrar líneas; se debe explicar el error de tabla mutante y cómo lo evita la construcción.*

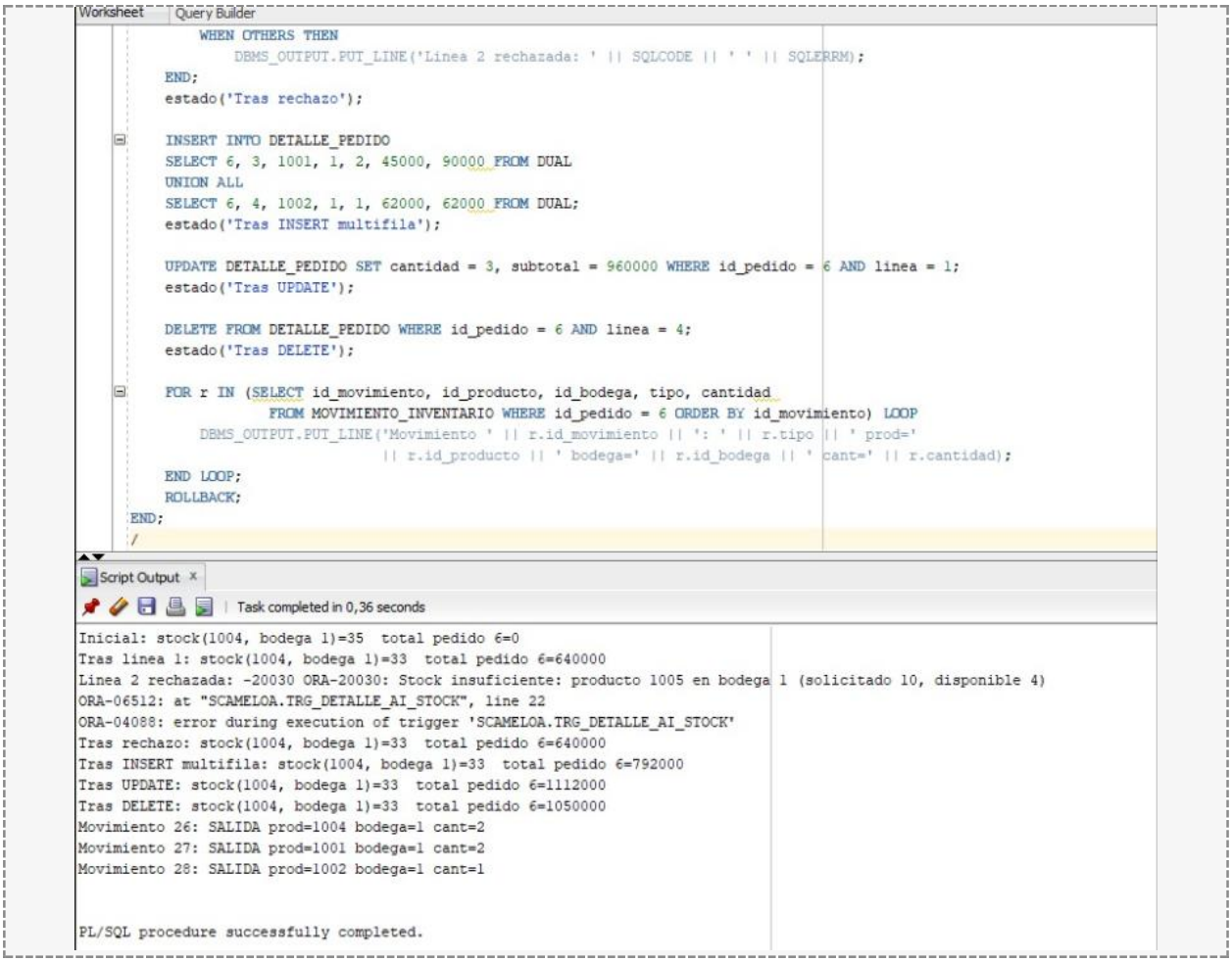
Explicación de la solución

El primer intento fue un trigger de fila que suma DETALLE_PEDIDO para actualizar PEDIDO. Falla en ejecución (no al crearlo) con ORA-04091: la tabla está mutando, es decir, la sentencia que dispara el trigger la está modificando y un trigger de fila no puede leerla, porque vería un estado intermedio (unas filas ya cambiadas y otras no).

El COMPOUND TRIGGER lo resuelve separando las fases: AFTER EACH ROW solo anota en una colección los pedidos afectados (sin leer la tabla) y AFTER STATEMENT, cuando la sentencia terminó y la tabla ya no está mutando, recalcula una vez cada pedido con SUM(subtotal). Al indexar la colección por id de pedido no se recalcula dos veces el mismo pedido, y antes de sumar se bloquea la cabecera para serializar sesiones concurrentes.

Evidencias de ejecución

Captura 5.5-a — Primer intento: trigger de fila que suma DETALLE_PEDIDO



Captura 5.5-b — COMPOUND TRIGGER: INSERT multifila, UPDATE y DELETE

```
SET SERVEROUTPUT ON SIZE UNLIMITED

DECLARE
    PROCEDURE sp_total (param_etiqueta VARCHAR2) IS
        vn_total NUMBER;
    BEGIN
        SELECT total INTO vn_total FROM PEDIDO WHERE id_pedido = 6;
        DBMS_OUTPUT.PUT_LINE(param_etiqueta || ': total del pedido 6 = ' || vn_total);
    END sp_total;
BEGIN
    sp_total('Inicial');

    INSERT INTO DETALLE_PEDIDO VALUES (6, 1, 1004, 1, 2, 320000, 640000);
    sp_total('Tras INSERT de 1 linea');

    INSERT INTO DETALLE_PEDIDO
    SELECT 6, 2, 1001, 1, 2, 45000, 90000 FROM DUAL
    UNION ALL
    SELECT 6, 3, 1002, 1, 1, 62000, 62000 FROM DUAL;
    sp_total('Tras INSERT multifila (2 lineas)');
```

Script Output

Task completed in 0,515 seconds

```
Inicial: total del pedido 6 = 0
Tras INSERT de 1 linea: total del pedido 6 = 640000
Tras INSERT multifila (2 lineas): total del pedido 6 = 792000
Tras UPDATE de la linea 1: total del pedido 6 = 1112000
Tras DELETE de la linea 3: total del pedido 6 = 1050000


PL/SQL procedure successfully completed.


Rollback complete.
```

5.6 TRG_V_STOCK_IO (INSTEAD OF) — 1 punto

Enunciado: *INSTEAD OF UPDATE sobre V_STOCK_CONSOLIDADO que actualiza el precio y el stock mínimo del producto subyacente; se debe explicar por qué la vista no es actualizable.*

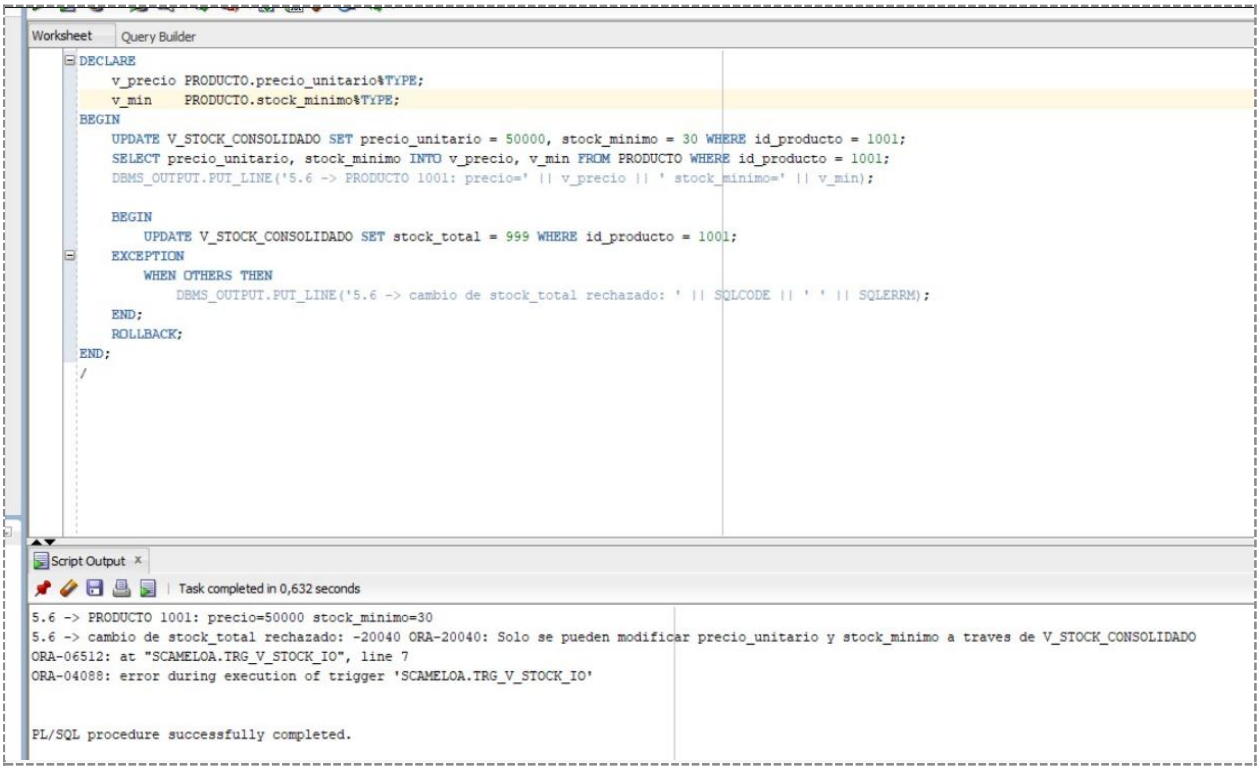
Explicación de la solución

La vista no es actualizable directamente porque agrega con GROUP BY y SUM sobre un LEFT JOIN: una fila de la vista resume varias filas base y Oracle no puede traducir el UPDATE a filas concretas (ORA-01732). El trigger INSTEAD OF UPDATE, que es de fila, reemplaza la operación y actualiza PRODUCTO (precio_unitario y stock_minimo) por id_producto; los demás triggers (margen y auditoría de precio) se siguen aplicando. Si se intenta cambiar otra columna de la vista se rechaza con -20040, en lugar de ignorarla en silencio.

Evidencias de ejecución

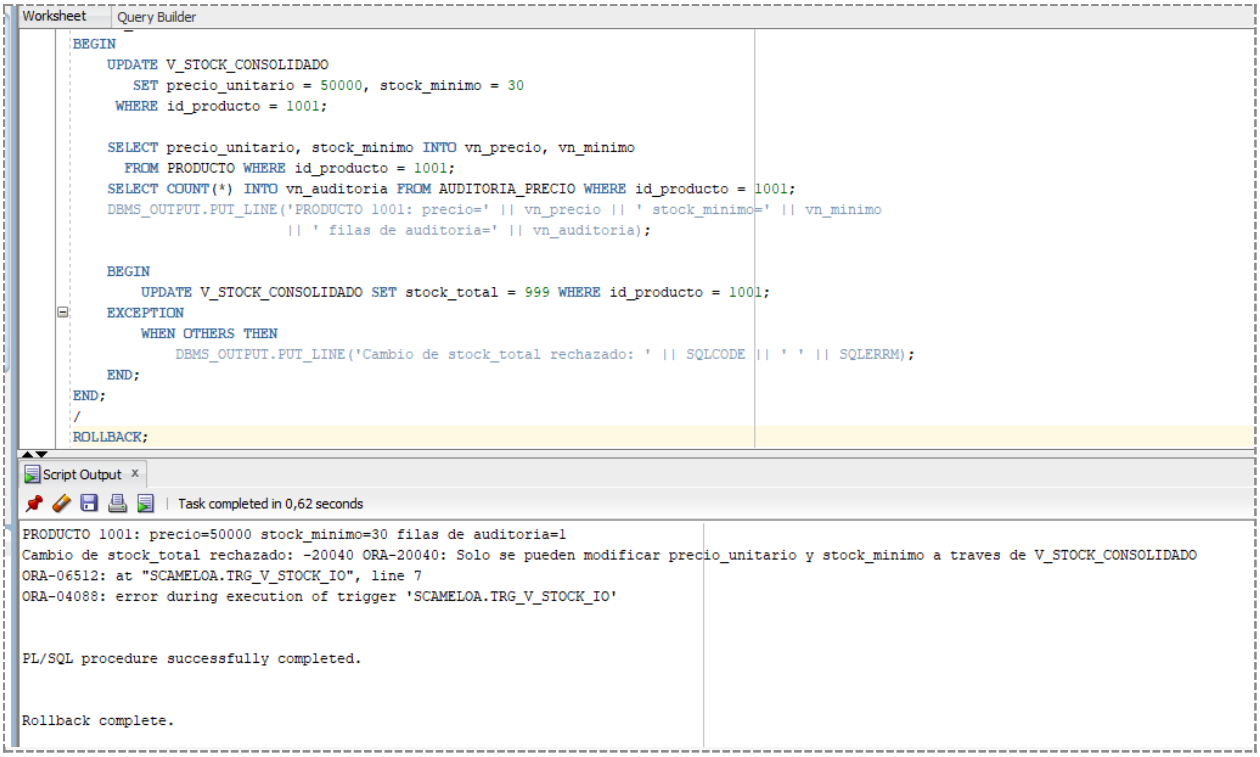
Captura 5.6-a — UPDATE directo sobre la vista (trigger deshabilitado)

Ejecutar: evidencias_pruebas.sql › E-5.6-a
Debe verse: ORA-01732: operación DML no válida en esta vista.



Captura 5.6-b — UPDATE a través de la vista con el trigger INSTEAD OF

.



Parte 6 — Caso integrador (5 puntos)

6 PKG_VENTAS.SP_FACTURAR_PEDIDO — 5 puntos

Enunciado: *Factura un pedido en BORRADOR integrando los objetos de las partes 1 a 5, con SAVEPOINT, registro de errores y parámetros OUT de total y descuento.*

Explicación de la solución

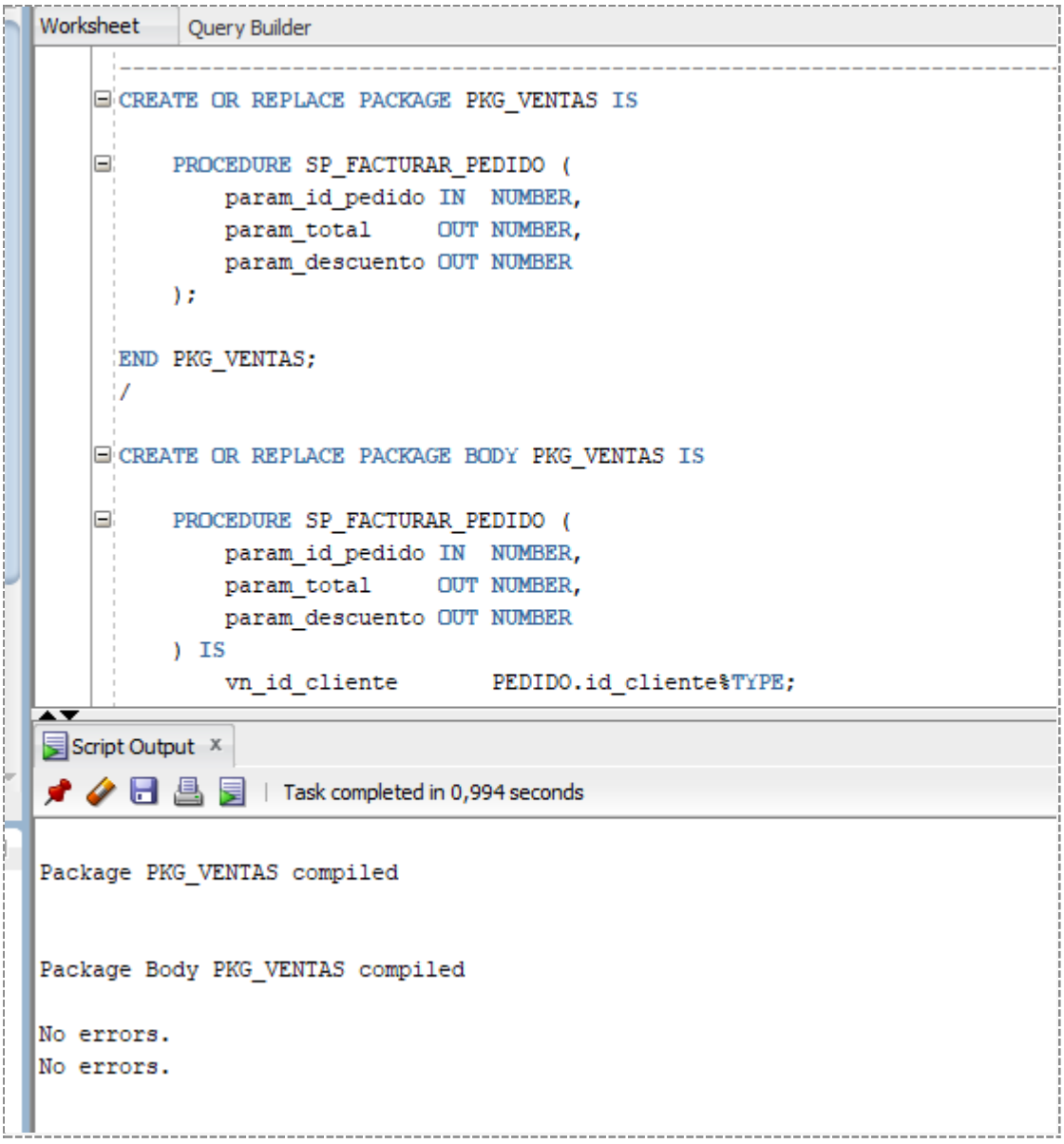
El procedimiento crea un SAVEPOINT, bloquea el pedido (FOR UPDATE) para impedir facturaciones concurrentes, valida que exista y esté en BORRADOR, acumula el subtotal con un cursor, obtiene el descuento con FN_CALCULAR_DESCUENTO y valida el cupo con FN_CUPO_DISPONIBLE, sin reimplementar su lógica. Luego actualiza el pedido (total y estado CONFIRMADO) y el saldo pendiente del cliente, y devuelve total y descuento por los parámetros OUT. También bloquea la fila del cliente antes de leer su saldo, para que dos pedidos del mismo cliente facturados a la vez no pasen la validación de cupo con el mismo saldo.

Todas las fallas usan RAISE_APPLICATION_ERROR con códigos -20050 a -20053 (fuera de los usados en las partes 1 a 5) y un único manejador revierte al savepoint, registra con SP_REGISTRAR_ERROR (autónomo: sobrevive al rollback) y relanza la misma excepción. Alcance transaccional: el paquete no hace COMMIT; la confirmación corresponde a quien lo invoca, porque facturar puede ser un paso de una operación mayor y solo el llamador sabe si debe confirmarse o descartarse completa. Lo único que el paquete revierte por sí mismo es su propio trabajo, hasta el savepoint.

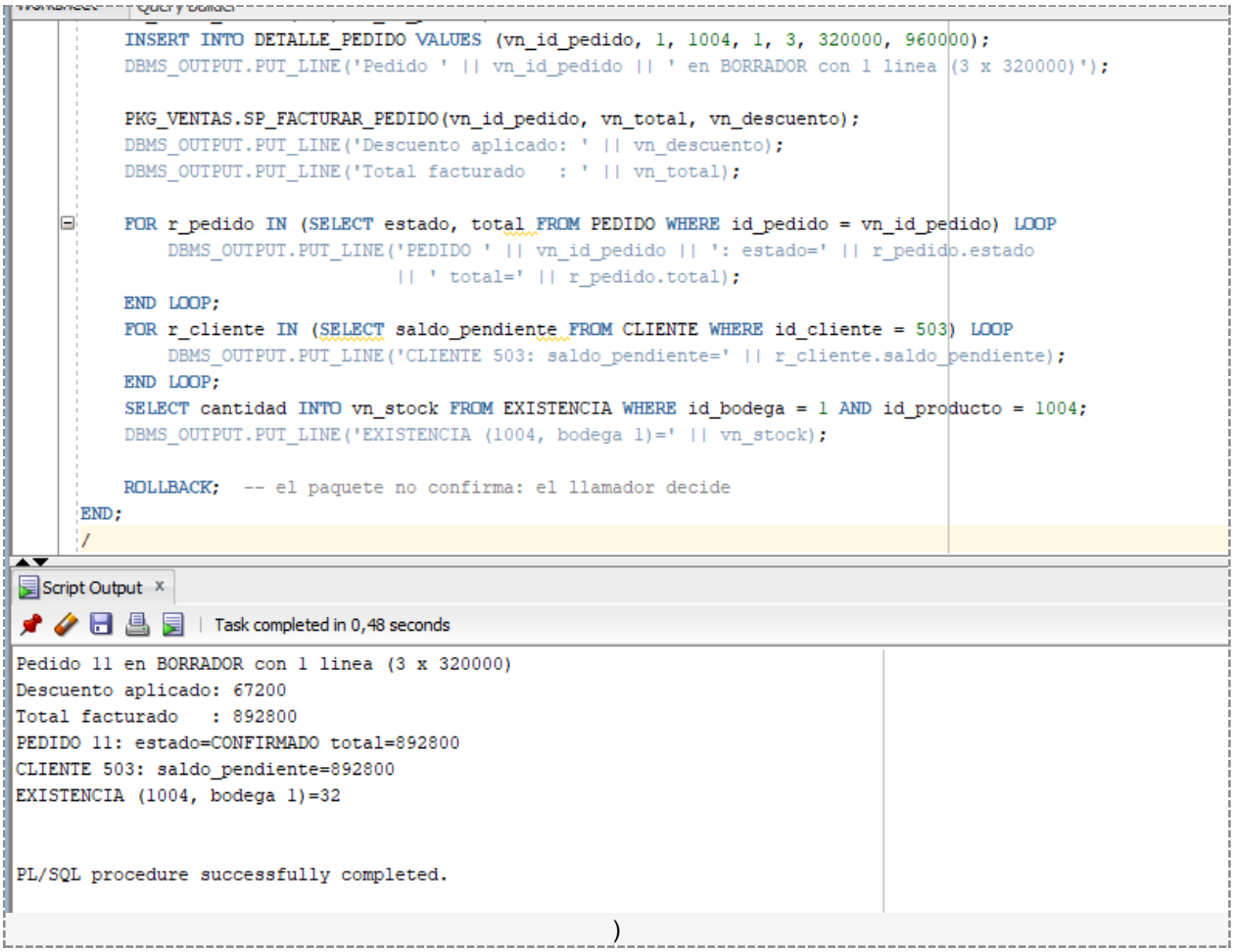
Evidencias de ejecución

Captura 6-a — Estado de la especificación y del cuerpo del paquete

Ejecutar: evidencias_pruebas.sql › E-6-a
Debe verse: PKG_VENTAS (PACKAGE y PACKAGE BODY) en estado VALID.



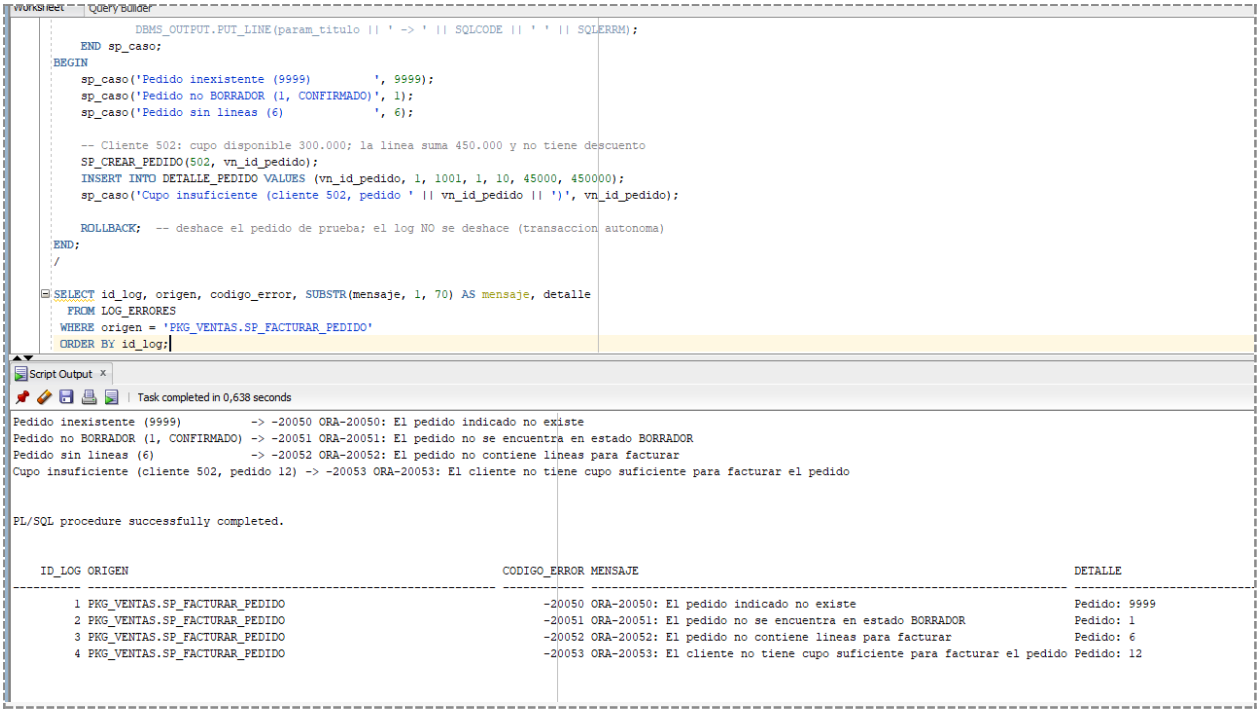
Captura 6-b — Facturación exitosa (cliente 503, 3 × 320000)



Captura 6-c — Errores controlados y registro persistente en LOG_ERRORES

Ejecutar: evidencias_pruebas.sql › E-6-c

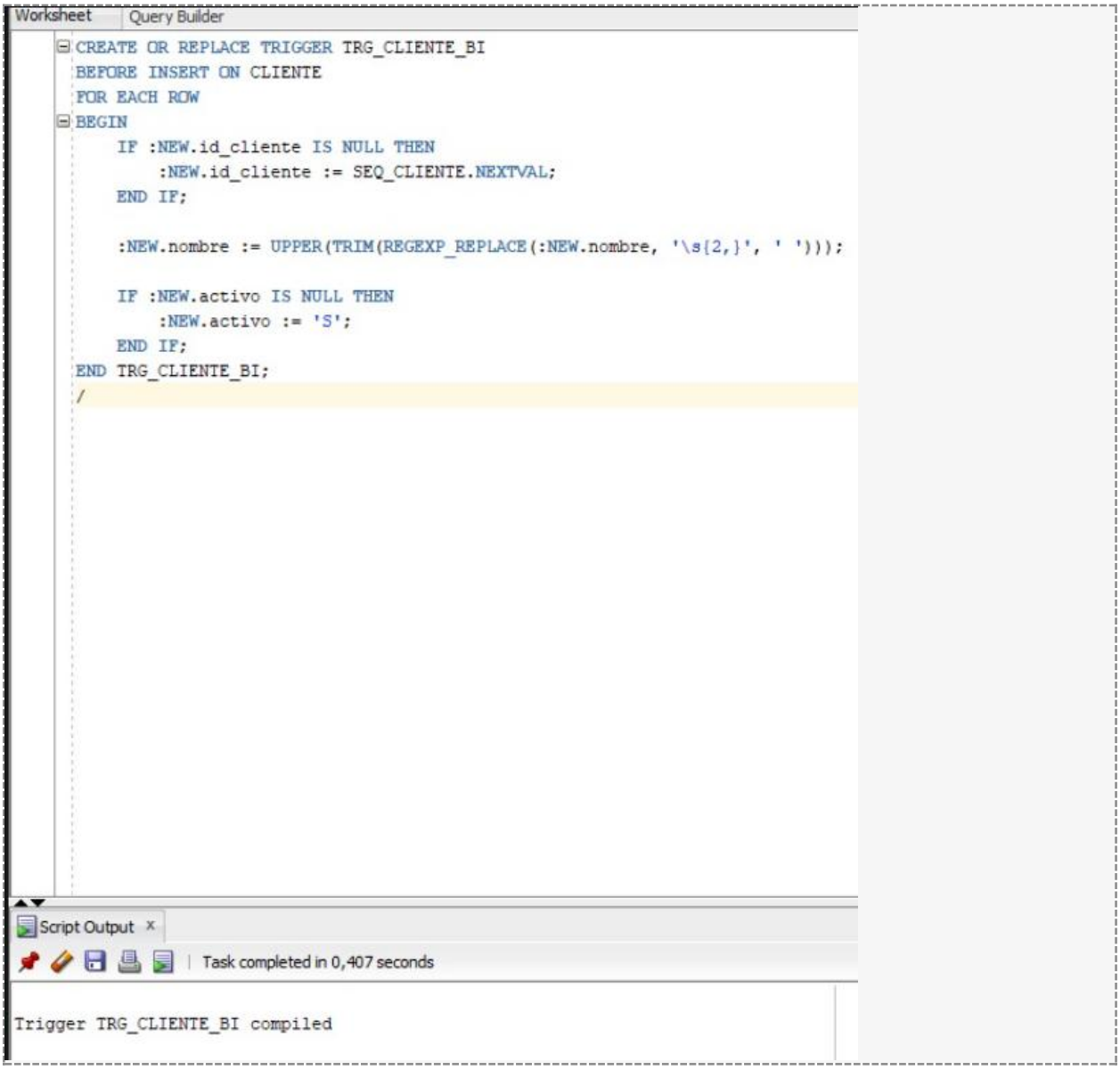
Debe verse: -20050, -20051, -20052 y -20053; cuatro filas en LOG_ERRORES que siguen después del ROLLBACK.



Anexo — Verificación general

Captura A-1 — Todos los objetos del taller en estado VALID

Ejecutar: evidencias_pruebas.sql › E-A1
Debe verse: Procedimientos, funciones, paquete y triggers con STATUS = VALID.



Captura A-2 — Triggers del taller habilitados

Ejecutar: evidencias_pruebas.sql › E-A2
Debe verse: Los seis triggers de la Parte 5 con STATUS = ENABLED.

